Forecasting Demand for National Research Supercomputing

Australian Academy of Science Internship Project

Yusuf Kurnia Romadhon

Table of contents

1 Forecasting Demand for National Research Supercomputing 5

2 Introduction 6
- 2.1 Project Background . . . 6
- 2.2 Project Purpose . . . 6
- 2.3 The Forecasting Problem . . . 7
- 2.4 Project Objectives . . . 7
- 2.5 Analysis Framework . . . 8

3 Data Collection 9
- 3.1 Purpose . . . 9
- 3.2 Data Sources . . . 9
  - 3.2.1 Year-specific HPC source summary . . . 10
- 3.3 HPC Demand and Allocation Data . . . 12
- 3.4 Field of Research Data . . . 13
- 3.5 Potential Demand Drivers . . . 14
- 3.6 Data Provenance . . . 14
- 3.7 Data Collection Limitations . . . 15

4 Variable Definition 16
- 4.1 Purpose . . . 16
- 4.2 HPC Demand . . . 16
  - 4.2.1 Requested Compute . . . 17
  - 4.2.2 Allocated Compute . . . 17
  - 4.2.3 Latent Demand . . . 17
- 4.3 Demand and Allocation Indicators . . . 17
- 4.4 Field of Research . . . 18
- 4.5 Potential Structural Drivers . . . 19
  - 4.5.1 Research Funding . . . 19
  - 4.5.2 Research Workforce and Activity . . . 19
  - 4.5.3 Compute Intensity . . . 19
  - 4.5.4 HPC Infrastructure . . . 20
- 4.6 Variable Selection for Modelling . . . 20

5 Data Integration **21**
5.1 Purpose . . . 21
5.2 Main Datasets . . . 21
5.3 Yearly Source Integration . . . 22
5.3.1 Load Libraries and Define Paths . . . 22
5.4 Annual Demand and Allocation Summary Construction . . . 29
5.4.1 Reconstruct Annual Demand and Allocation Summary . . . 29
5.4.2 Construct Reconstructed Annual Summary . . . 31
5.4.3 Compare Reconstructed and Existing Annual Summaries 36
5.4.4 Resolve Unreconstructed Annual Values . . . 37
5.4.5 Derive Missing Annual Measures . . . 38
5.4.6 Final Reconciliation Check . . . 41
5.4.7 Export Reconciled Annual Demand and Allocation Summary 43
5.5 Field of Research Summary Construction . . . 44
5.5.1 Assess Historical FoR Comparability . . . 46
5.5.2 Construct FoR-Level Compute Measures . . . 48
5.5.3 Export Reconciled Field of Research Summary . . . 51

6 Data Standardisation **53**
6.1 Purpose . . . 53
6.2 Reporting Period . . . 53
6.3 Institution Names . . . 54
6.4 Field of Research Classification . . . 54
6.4.1 Historical FoR Reporting Groups . . . 56
6.5 HPC Facilities and Systems . . . 59
6.6 Resource Type . . . 60
6.7 Units . . . 61
6.8 Data Quality and Comparability . . . 61
6.8.1 Data Provenance and Estimation . . . 61
6.8.2 Comparability Across Reporting Years . . . 62
6.8.3 Treatment for Subsequent Analysis . . . 62
6.9 Output . . . 62

7 Data Conversion **64**
7.1 Purpose . . . 64
7.2 Conversion Approach . . . 65
7.3 CPU Resources . . . 73
7.4 GPU Resources . . . 84
7.4.1 Historical GPU Estimation Scenarios . . . 94
7.5 Field of Research Conversion . . . 102
7.6 Output . . . 109

8 Initial Analysis **112**
8.1 Purpose . . . 112
8.2 Historical Demand and Allocation . . . 113
8.2.1 Annual Data Validation . . . 113

8.2.2 Annual Analytical Indicators . . . 114
8.2.3 Annual Demand and Allocation Trends . . . 115
8.2.4 Initial Findings . . . 117
8.3 CPU and GPU Resources . . . 117
8.3.1 Validate the converted CPU and GPU series . . . 118
8.3.2 Create CPU and GPU trend charts . . . 119
8.3.3 Initial Findings . . . 124

9 Model Development and Benchmarking 126
9.1 Purpose . . . 126
9.2 Forecast Target . . . 126
9.3 Benchmark Models . . . 127
9.4 Structural Demand Model . . . 127
9.5 Leading Indicators . . . 127
9.6 Variable Selection . . . 128
9.7 Model Benchmarking . . . 128
9.8 CPU and GPU Modelling . . . 128
9.9 Modelling Limitations . . . 129
9.10 Output . . . 129

10 Model Evaluation 130
10.1 Purpose . . . 130
10.2 Evaluation Approach . . . 130
10.3 Time-Series Cross-Validation . . . 130
10.4 Forecast Accuracy . . . 131
10.4.1 Mean Absolute Error . . . 131
10.4.2 Root Mean Squared Error . . . 131
10.4.3 Mean Absolute Percentage Error . . . 131
10.5 Residual Diagnostics . . . 131
10.6 Forecast Behaviour . . . 132
10.7 Structural Model Evaluation . . . 132
10.8 Comparison with Benchmark Models . . . 132
10.9 CPU and GPU Evaluation . . . 133
10.10 Evaluation Limitations . . . 133
10.11 Output . . . 133

11 Model Selection 134
11.1 Purpose . . . 134
11.2 Selection Criteria . . . 134
11.3 Benchmark and Structural Models . . . 134
11.4 Structural Driver Selection . . . 135
11.5 CPU and GPU Modelling Approach . . . 135
11.6 Expert Review . . . 135
11.7 Final Model Selection . . . 136
11.8 Output . . . 136

12 Scenario Forecasting and Capacity-Gap Analysis **137**
12.1 Purpose . . . 137
12.2 Forecast Horizon . . . 137
12.3 Scenario Framework . . . 137
12.4 Status Quo Scenario . . . 138
12.5 Restored National R&D Intensity Scenario . . . 138
12.6 AI Acceleration Scenario . . . 138
12.7 CPU and GPU Forecasts . . . 138
12.8 Capacity Comparison . . . 139
12.9 Facility-Equivalent Capacity . . . 139
12.10 Scenario Comparison . . . 139
12.11 Uncertainty and Sensitivity . . . 139
12.12 Policy Interpretation . . . 140
12.13 Output . . . 140

13 References and AI Acknowledgement **141**
13.1 References . . . 141
13.2 AI Acknowledgement . . . 143

Chapter 1

Forecasting Demand for National Research Supercomputing

Australian Academy of Science Internship Project

Chapter 2

Introduction

2.1 Project Background

High-performance computing (HPC) is an important component of Australia's research infrastructure, supporting computationally intensive research across a wide range of scientific fields (Australian Academy of Science 2024).

Australia has two national Tier-1 HPC facilities accessible to researchers: the National Computational Infrastructure (NCI) and the Pawsey Supercomputing Research Centre (National Computational Infrastructure, n.d.; Pawsey Supercomputing Research Centre, n.d.).

As research becomes increasingly data- and compute-intensive, understanding how demand for HPC resources is changing is important for planning Australia's future national research infrastructure (Australian Academy of Science 2024).

2.2 Project Purpose

The purpose of this project is to analyse historical Australian research computing demand and develop an evidence-based projection of future HPC demand to 2030 (Australian Academy of Science 2026).

The analysis is intended to support national policy discussions relating to public compute investment and future research infrastructure planning (Australian Academy of Science 2026).

The project combines data engineering, descriptive analysis and forecasting using publicly available NCI and Pawsey data, particularly information from the National Computational Merit Allocation Scheme (NCMAS), together with additional indicators where they are sufficiently consistent and relevant (Australian Academy of Science 2026).

Because historical reporting practices, hardware systems and compute accounting methods have changed over time, the analysis explicitly documents assumptions, derived values and limitations rather than treating all historical observations as directly comparable.

2.3 The Forecasting Problem

One of the main challenges in forecasting national research computing demand is that observed HPC allocation does not necessarily represent the full level of research demand.

National allocation schemes operate under finite capacity. As a result, allocated compute represents the amount of resource awarded rather than necessarily the amount researchers would use if additional capacity were available (Australian Academy of Science 2026).

Requested compute provides additional evidence of demand, but it may also understate underlying need where researchers constrain their requests based on expected availability or where potential users do not apply (Australian Academy of Science 2026).

The project therefore distinguishes between:

- requested compute,
- allocated compute,
- the gap between requested and allocated compute, and
- broader underlying or latent demand, which cannot be observed directly from the available allocation data.

A further challenge is that the historical series is not fully comparable across years. Reporting practices, hardware systems, compute units and service-unit accounting have changed over time, and CPU and GPU resources are not directly interchangeable (Australian Academy of Science 2026).

For this reason, the analysis uses the longest available historical series while explicitly documenting where values are directly reported, derived, estimated or based on assumptions. Forecasting results are therefore interpreted together with these data limitations rather than as precise measurements of unconstrained demand.

2.4 Project Objectives

The project aims to:

1. build a documented longitudinal dataset of Australian national HPC activity using publicly available NCI and Pawsey sources, including NCMAS allocation information,

2. document differences in reporting practices, compute units, systems and accounting methods across years,
3. improve comparability of historical compute measures where a defensible conversion or normalisation approach is available,
4. distinguish CPU and GPU resources where the available source data permits,
5. examine historical patterns in requested compute, allocation and oversubscription,
6. analyse changes in Field of Research composition using the most comparable available data,
7. assess potential indicators or structural drivers of research computing demand where suitable data are available,
8. develop and compare forecasting or projection approaches appropriate to the length and quality of the historical series, and
9. develop demand scenarios to 2030 and assess their implications for future national research computing requirements.

Throughout the analysis, assumptions, derived values, source limitations and uncertainty are documented explicitly.

The project objectives are informed by the internship brief, which emphasises longitudinal data assembly, descriptive analysis, structural demand modelling and scenario development (Australian Academy of Science 2026).

2.5 Analysis Framework

The analysis follows ten main steps:

1. Data collection
2. Variable definition
3. Data integration
4. Data standardisation
5. Data conversion and comparability assessment
6. Exploratory and descriptive analysis
7. Model development and comparison
8. Model evaluation
9. Model selection
10. Scenario forecasting and interpretation

Chapter 3

Data Collection

3.1 Purpose

The purpose of the data collection stage is to assemble a documented longitudinal evidence base for analysing Australian national research computing demand and allocation.

The project draws primarily on publicly available NCI, Pawsey and NCMAS reporting to identify historical requested compute, allocated compute, resource type, Field of Research and other available characteristics of national HPC allocations and usage (Australian Academy of Science 2026)

Additional datasets, including research funding, workforce and research activity indicators, are collected as potential structural drivers of future HPC demand where sufficiently consistent time-series data are available (Australian Academy of Science 2026).

Because the availability, scope and definition of variables differ substantially across years and sources, the purpose of this stage is not to assume that all collected observations are directly comparable. Instead, original values, units, reporting periods, source definitions and data limitations are retained so that their suitability for integration, standardisation and modelling can be assessed in subsequent stages.

3.2 Data Sources

The project uses multiple public data sources from NCI, Pawsey and NCMAS, together with research funding, workforce and Field of Research datasets.

The specific HPC source varies by analytical year. Earlier observations are primarily drawn from annual reports, allocation documents and other historical

NCI and Pawsey publications, while more recent observations increasingly use direct NCMAS outcome pages, applicant guidance and facility-specific allocation information.

In several years, different documents are used for different analytical components. Allocation outcomes, requested-demand information, resource availability and Field of Research composition may therefore originate from separate official publications.

For transparency and reproducibility, the exact source document, source location and URL are retained in the underlying yearly datasets. The principal sources used for each analytical year are summarised below.

3.2.1 Year-specific HPC source summary

```
library(tibble)
library(knitr)

source_table <- tribble(
  ~Year, ~`Demand / Context Source`, ~`Allocation Source`, ~`FoR Source`,

  2014,
  "NCI Annual Report 2013/14; NCMAS Information for Applicants 2014",
  "NCI Annual Report 2013/14",
  "NCI Annual Report 2013/14",

  2015,
  "NCI Annual Report 2014/15",
  "NCI Annual Report 2014/15; Pawsey Annual Report 2014/15",
  "NCI Annual Report 2014/15",

  2016,
  "NCI Annual Report 2015/16; CSIRO Annual Report 2015/16",
  "NCI Annual Report 2015/16",
  "NCI Annual Report 2015/16",

  2017,
  "NCI Annual Report 2016/17; Pawsey Annual Report 2016/17",
  "NCI Annual Report 2016/17; Pawsey Annual Report 2016/17",
  "Pawsey Annual Report 2016/17",

  2018,
  "NCI Annual Report 2017/18; NCMAS 2018 results",
  "NCI Annual Report 2017/18",
  "NCI Annual Report 2017/18",
```

```
  2019,
  "NCI Annual Report 2018/19; Pawsey 2019 NCMAS information",
  "NCI Annual Report 2018/19",
  "NCI Annual Report 2018/19",

  2020,
  "NCI Annual Report 2019/20; NCMAS 2020 Information for Applicants",
  "NCI 2019/20 Compute Allocations Detail",
  "NCI 2019/20 Compute Allocations Detail",

  2021,
  "NCMAS 2021 Outcomes; NCI and Pawsey annual reports",
  "NCMAS 2021 Outcomes; NCI Compute Allocations 2020/21",
  "NCMAS 2021 Outcomes",

  2022,
  "NCMAS 2023 Information for Applicants; NCI/Pawsey reports",
  "NCMAS 2022 Outcomes",
  "NCMAS 2022 Outcomes",

  2023,
  "NCMAS 2023 Outcomes; NCI/Pawsey supporting publications",
  "NCMAS 2023 Outcomes",
  "NCMAS 2023 Outcomes",

  2024,
  "NCMAS 2024 Outcomes; NCMAS 2024 Information for Applicants",
  "NCMAS 2024 Outcomes",
  "NCMAS 2024 Outcomes",

  2025,
  "NCMAS 2025 Outcomes; Pawsey 2025 Allocations",
  "NCMAS 2025 Outcomes",
  "NCMAS 2025 Outcomes",

  2026,
  "NCMAS 2026 Outcomes; NCMAS 2026 Information for Applicants",
  "NCMAS 2026 Outcomes",
  "NCMAS 2026 Outcomes"
)

knitr::kable(
  source_table,
  format = "html",
  escape = TRUE,
```

```
  align = c("c", "l", "l", "l")
)
```

Table 3.1: Principal year-specific sources used to constr
dataset

Year	Demand / Context Source	Allocation Source
2014	NCI Annual Report 2013/14; NCMAS Information for Applicants 2014	NCI Annual Report 2013/14
2015	NCI Annual Report 2014/15	NCI Annual Report 2014/15
2016	NCI Annual Report 2015/16; CSIRO Annual Report 2015/16	NCI Annual Report 2015/16
2017	NCI Annual Report 2016/17; Pawsey Annual Report 2016/17	NCI Annual Report 2016/17
2018	NCI Annual Report 2017/18; NCMAS 2018 results	NCI Annual Report 2017/18
2019	NCI Annual Report 2018/19; Pawsey 2019 NCMAS information	NCI Annual Report 2018/19
2020	NCI Annual Report 2019/20; NCMAS 2020 Information for Applicants	NCI 2019/20 Compute Alloc
2021	NCMAS 2021 Outcomes; NCI and Pawsey annual reports	NCMAS 2021 Outcomes; NC
2022	NCMAS 2023 Information for Applicants; NCI/Pawsey reports	NCMAS 2022 Outcomes
2023	NCMAS 2023 Outcomes; NCI/Pawsey supporting publications	NCMAS 2023 Outcomes
2024	NCMAS 2024 Outcomes; NCMAS 2024 Information for Applicants	NCMAS 2024 Outcomes
2025	NCMAS 2025 Outcomes; Pawsey 2025 Allocations	NCMAS 2025 Outcomes
2026	NCMAS 2026 Outcomes; NCMAS 2026 Information for Applicants	NCMAS 2026 Outcomes

Key source families include NCI annual and allocation reports (National Computational Infrastructure 2014a, 2015, 2016, 2017, 2018a, 2019, 2020a, 2020b, 2021), Pawsey annual and allocation publications (Pawsey Supercomputing Research Centre 2015, 2017, 2019), and NCMAS outcomes and applicant guidance (National Computational Infrastructure 2014b, 2018b, 2020c, 2022, 2023, 2025; National Computational Merit Allocation Scheme 2021, 2022, 2023, 2024, 2025, 2026).

3.3 HPC Demand and Allocation Data

The main HPC dataset covers source periods from 2013/14 to 2025/26, corresponding to analytical years 2014 to 2026.

The collected information includes, where available:

- requested compute,
- allocated compute,
- facility and computing system,
- resource type,
- original compute unit,
- Field of Research,
- institution,
- allocation or reporting scope,

- source document and URL, and
- source reporting period.

The availability and level of detail of these variables vary substantially across years. Earlier observations are often available only as aggregate values reported in annual reports, while more recent NCMAS outcomes provide more detailed project-, facility- or resource-level information.

Requested and allocated compute are retained as distinct measures. Allocated compute represents resources awarded through the relevant allocation process, while requested compute provides additional evidence of demand where it is directly reported or can be defensibly derived.

Historical values are also retained in their original reported units before any conversion or standardisation is applied. This is important because service units, CPU hours, GPU hours and other compute measures are not directly interchangeable across facilities, systems or hardware generations (Australian Academy of Science 2026).

Where a value is not directly reported, its derivation or assumption is recorded separately so that directly reported, derived and proxy values can be distinguished during subsequent analysis.

3.4 Field of Research Data

Field of Research (FoR) information was collected to examine how the disciplinary composition of national HPC allocations has changed over time.

The source, granularity and reliability of FoR information vary across years. Earlier observations are derived from annual reports, allocation summaries or project-level information with varying levels of detail, while more recent NCMAS outcomes provide more direct and consistent FoR information.

Where possible, reported FoR classifications are mapped to a common classification to support comparison across years. The original classification and source-reported FoR information are retained so that the standardisation process remains traceable.

The available FoR data primarily represents the composition of allocated compute rather than the composition of requested or latent demand. FoR allocation shares are therefore not interpreted directly as FoR-specific research demand.

Where allocation composition is subsequently combined with annual requested-compute totals to estimate demand by FoR, the resulting values are treated as analytical proxies rather than directly observed demand.

Because source scope and data quality differ across years, longer-term FoR trends are interpreted with caution, with the most confidence placed in years where direct and more comparable NCMAS FoR information is available.

3.5 Potential Demand Drivers

Additional datasets are collected to investigate potential structural drivers of national research computing demand.

These include research funding, research workforce, research activity and related indicators that may help explain changes in HPC demand over time. The internship brief identifies competitive grant flows, research workforce size and changes in compute intensity as potential structural drivers of future demand (Australian Academy of Science 2026).

These variables are treated as candidate explanatory indicators rather than assumed drivers. Their suitability for modelling is assessed later based on historical coverage, consistency, temporal alignment and their observed relationship with the available HPC demand measures.

Variables with insufficient coverage or weak comparability are retained for contextual analysis where useful but are not necessarily included in the final forecasting model.

3.6 Data Provenance

Data provenance is retained throughout the dataset so that each observation can be traced to its originating source information and, where relevant, the derivation used to construct it.

Where available, the retained provenance information includes:

- source document or webpage,
- source URL,
- source reporting period,
- source location or relevant section,
- originally reported value and unit,
- reporting or allocation scope, and
- notes describing extraction, derivation or interpretation.

This information provides an audit trail for subsequent data integration, standardisation and conversion. It also makes it possible to distinguish values that are directly reported from those that are derived, estimated or based on analytical assumptions.

Maintaining provenance is particularly important because the historical HPC series is assembled from publications that differ in reporting format, scope, compute units and level of detail across years. Provenance information is therefore used to assess whether observations can be compared directly or require additional qualification before analysis.

3.7 Data Collection Limitations

Several limitations were identified during the data collection stage.

The main limitations include:

- limited availability of sufficiently consistent historical data before 2013,
- requested compute may understate underlying demand where researchers constrain applications in response to expected resource availability,
- changes in reporting formats, terminology and level of detail across years,
- inconsistent availability of requested and allocated compute,
- incomplete separation of CPU and GPU resources in some historical records,
- changes in HPC systems, hardware generations and accounting methods,
- differences in reporting scope between NCI, Pawsey and NCMAS sources,
- changes in Field of Research classifications and reporting granularity, and
- the use of derived or proxy values where directly reported measures are not available.

These limitations mean that the historical series cannot be treated as perfectly homogeneous or directly comparable across all years.

The impact of these issues is assessed in later stages of the project through data integration, standardisation, conversion and comparability assessment. Observations that require assumptions or additional qualification are retained with supporting provenance information rather than being treated as equivalent to directly reported values.

Chapter 4

Variable Definition

4.1 Purpose

The purpose of the variable definition stage is to establish clear and consistent meanings for the measures used throughout the analysis.

The variables are organised into three main groups:

- HPC demand variables, including requested compute and related demand indicators,
- allocation and infrastructure variables, including awarded compute and available capacity, and
- potential structural drivers that may help explain changes in research computing demand over time.

Not all defined variables are expected to enter the final forecasting model. Some are used primarily for descriptive analysis, comparability assessment or interpretation.

Variables considered for modelling are evaluated later based on their theoretical relevance, historical coverage, consistency of measurement, temporal alignment and suitability for the relatively short annual time series available.

4.2 HPC Demand

The main outcome of interest is Australian research demand for national HPC resources.

Demand is not directly observed through a single consistent measure. The internship brief notes that national HPC allocations are supply-constrained and that researchers may also limit requests based on expected availability,

meaning that observed requests may not capture the full level of underlying need (Australian Academy of Science 2026).

The analysis therefore distinguishes between requested compute, allocated compute and underlying or latent demand.

4.2.1 Requested Compute

Requested compute represents the amount of HPC resource requested by researchers through an allocation process where this information is available.

It is treated as the primary observable indicator of research demand. However, requested compute may understate underlying need if researchers limit their requests based on expected resource availability or if potential users do not submit applications (Australian Academy of Science 2026).

4.2.2 Allocated Compute

Allocated compute represents the amount of HPC resource awarded to researchers through the relevant allocation process, as reported in NCMAS outcomes and related NCI and Pawsey allocation documentation (National Computational Merit Allocation Scheme 2024, 2025, 2026).

Allocation is analysed separately from demand because the amount awarded is constrained by available resources and the allocation process.

4.2.3 Latent Demand

Latent demand refers in this project to the broader level of HPC resources that researchers might require beyond what is observed in submitted requests or awarded allocations.

This is an analytical concept rather than a directly observed variable. The internship brief highlights that observed demand may be censored by available capacity and by researchers adapting their requests to expected resource availability (Australian Academy of Science 2026).

4.3 Demand and Allocation Indicators

Several indicators are derived from the historical demand and allocation data to describe the relationship between requested and awarded compute.

The observed request-allocation gap is defined as:

$$\text{Request-Allocation Gap}_t = \text{Requested Compute}_t - \text{Allocated Compute}_t$$

A positive value indicates that reported requests exceeded the amount of compute allocated in year (t).

The fulfilment rate is defined as:

$$\text{Fulfilment Rate}_t = \frac{\text{Allocated Compute}_t}{\text{Requested Compute}_t}$$

A fulfilment rate below 1 indicates that allocated compute was lower than reported requested compute, while a value closer to 1 indicates that a larger proportion of reported requests was satisfied.

These indicators are used to describe allocation pressure over time. However, they are not interpreted as direct measures of total unmet research demand because requested compute may itself be constrained by expected resource availability and participation in the allocation process (Australian Academy of Science 2026).

The indicators are calculated only where requested and allocated compute refer to sufficiently comparable scope and resource definitions.

4.4 Field of Research

Field of Research (FoR) variables are used to describe the disciplinary composition of national HPC allocation.

The principal FoR variables are:

- `for_classification`: the standardised research field assigned to an observation,
- `allocation_share_for`: the proportion of annual allocated compute associated with each FoR,
- `allocated_compute_for`: allocated compute associated with each FoR, and
- `estimated_demand_by_for_proxy`: an analytical estimate of requested compute by FoR where allocation composition is applied to annual requested compute.

The FoR allocation share for field (i) in year (t) is defined as:

$$\text{Allocation Share}_{i,t} = \frac{\text{Allocated Compute}_{i,t}}{\text{Total Allocated Compute}_t}$$

Where allocation composition is applied to annual requested compute, the resulting FoR-level demand measure is explicitly treated as a proxy rather than directly observed demand.

This distinction is maintained because the available FoR information primarily describes allocation composition rather than the composition of submitted requests.

4.5 Potential Structural Drivers

Potential structural drivers are candidate explanatory variables that may help describe changes in national HPC demand over time (Australian Academy of Science 2026).

They are not assumed to have a causal relationship with HPC demand and are considered for modelling only where historical coverage and measurement consistency are sufficient.

4.5.1 Research Funding

Research funding variables include available annual measures from:

- Australian Research Council (ARC),
- National Health and Medical Research Council (NHMRC), and
- Medical Research Future Fund (MRFF).

These variables are considered as indicators of changes in the scale of funded research activity that may be associated with research computing requirements.

4.5.2 Research Workforce and Activity

Research workforce and activity variables include available measures of:

- research workforce size, and
- research activity.

These variables are considered as indicators of changes in the scale of the Australian research system.

4.5.3 Compute Intensity

Compute-intensity indicators describe changes in the amount or type of computing associated with research activity.

Potential indicators include:

- compute allocation per project,
- compute allocation by Field of Research,
- changes in CPU allocation, and
- changes in GPU allocation.

These measures are used primarily to investigate whether the composition and intensity of research computing are changing over time.

4.5.4 HPC Infrastructure

HPC infrastructure variables describe changes in the supply environment within which observed demand and allocation occur.

Relevant variables may include:

- computing system,
- resource type,
- available compute capacity, and
- major system commissioning or retirement.

Infrastructure variables are interpreted separately from demand because an increase in allocated compute may reflect additional available capacity rather than an equivalent increase in underlying research demand.

4.6 Variable Selection for Modelling

Not all variables defined in this chapter will be included in the final forecasting model.

Candidate variables will be assessed based on:

- theoretical relevance to HPC demand,
- historical data coverage,
- consistency of measurement,
- temporal alignment with the demand series,
- observed relationship with the selected demand measure,
- availability or defensible assumptions over the forecast horizon, and
- the limited number of annual observations available for modelling.

Given the short annual time series, the final modelling approach will prioritise a small number of interpretable variables rather than a large set of predictors. Variables that are useful for descriptive analysis or interpretation may therefore be retained without being included in the final forecasting model.

Chapter 5

Data Integration

5.1 Purpose

The purpose of the data integration stage is to combine the different datasets collected for the project into a structured analytical dataset while preserving their original values, units, scope and provenance.

The source datasets cover different reporting periods, levels of granularity and data structures. Integration is therefore required to align common identifiers such as analytical year, facility, computing system, resource type and Field of Research before the data can be analysed together.

At this stage, integration focuses on organising and linking the available information rather than forcing observations into common units or classifications. Differences in naming, classification and compute units are addressed subsequently through data standardisation and conversion.

Where datasets cannot be combined directly because of differences in scope or granularity, those distinctions are retained explicitly rather than assuming equivalence.

5.2 Main Datasets

The integration process combines several datasets covering 2014 - 2026, with different analytical roles.

The core HPC datasets include:

- yearly HPC demand and allocation source workbooks,
- Field of Research composition data, and
- HPC system and capacity information.

Supporting datasets include:

- research funding data, and
- research workforce and research activity data.

The core datasets provide the historical evidence used to construct measures of requested compute, allocated compute and disciplinary composition. Supporting research funding, workforce and research activity datasets are retained separately for later integration into the modelling stage as candidate explanatory variables.

Because these datasets differ in temporal coverage, granularity and reporting scope, they are not merged indiscriminately. Integration is performed only where suitable common identifiers and compatible levels of aggregation are available.

Where annual demand or allocation measures are not directly available, derived or proxy values are constructed later from the integrated source evidence using explicitly documented assumptions. These selected annual measures are subsequently assembled into a separate summary dataset for modelling.

5.3 Yearly Source Integration

The detailed yearly source files are integrated before constructing the annual modelling summary.

For the HPC demand and allocation data, each yearly workbook contains multiple types of evidence. The integration therefore preserves the different analytical levels rather than reducing each workbook immediately to a single annual value.

Three recurring components are retained:

- `Project_Allocations`, containing project or allocation-level records,
- `Context_Metrics`, containing reported or derived annual and contextual measures, and
- `Resource_Envelope`, containing information on available resources and system-level capacity.

Field of Research data are integrated separately from the yearly `FoR_Composition` sheets.

The source filename and source period are retained during integration so that each observation remains traceable to its original yearly workbook. The resulting integrated tables form the evidence base from which annual summary measures are selected or derived later.

5.3.1 Load Libraries and Define Paths

```
library(tidyverse)
library(readxl)
```

```
demand_path <- "assets/data/raw data/demand and allocation"
for_path <- "assets/data/raw data/field of research"

clean_path <- "assets/data/data clean"
```

The yearly demand and allocation workbooks are identified from the raw data directory.

```
demand_files <- list.files(
  demand_path,
  pattern = "\\.xlsx$",
  full.names = TRUE
)
```

For the demand and allocation workbooks, the reporting period is derived from the source filename. A helper function is also defined to convert financial-year-style source periods to their ending analytical year.

```
get_source_period <- function(path) {

  filename <- basename(path)

  years <- str_match(
    filename,
    "(\\d{4})_(\\d{2})"
  )

  paste0(
    years[, 2],
    "/",
    years[, 3]
  )
}

get_end_year <- function(source_period) {

  end_year <- str_extract(
    source_period,
    "(?<=/)\\d{2}$"
  )

  2000 + as.numeric(end_year)
}
```

Each yearly demand and allocation workbook contains several recurring worksheets. The same set of yearly workbooks is therefore used to extract each analytical component separately.

5.3.1.1 Integrate Context Metrics

Demand and allocation context metrics are combined across the yearly workbooks.

These records contain annual or supporting measures used to interpret requested compute, allocated compute and related demand-allocation relationships.

```
context_metrics <- map_dfr(
  demand_files,
  function(file) {

    read_excel(
      file,
      sheet = "Context_Metrics",
      col_types = "text"
    ) |>
      mutate(
        source_file = basename(file),
        source_period = get_source_period(file)
      )
  }
)
```

5.3.1.2 Integrate Project Allocations

Project-level allocation records are combined across the yearly workbooks while preserving their original source-level structure.

```
project_allocations <- map_dfr(
  demand_files,
  function(file) {

    read_excel(
      file,
      sheet = "Project_Allocations",
      col_types = "text"
    ) |>
      mutate(
        source_file = basename(file),
        source_period = get_source_period(file)
      )
  }
)
```

5.3.1.3 Integrate Resource Envelope

Resource-envelope records are combined across the yearly workbooks.

These records describe available HPC systems, resource availability and capacity-related information used to interpret the supply environment associated with observed demand and allocation.

```
resource_envelope <- map_dfr(
  demand_files,
  function(file) {

    read_excel(
      file,
      sheet = "Resource_Envelope",
      col_types = "text"
    ) |>
      mutate(
        source_file = basename(file),
        source_period = get_source_period(file)
      )
  }
)
```

5.3.1.4 Align Analytical Year

A common analytical year is created to align observations across the different yearly source tables. The ending year of each source period is used as the common reference year where a consistent NCMAS or allocation year is not available.

```
context_metrics <- context_metrics |>
  mutate(
    year = get_end_year(source_period)
  )

project_allocations <- project_allocations |>
  mutate(
    year = coalesce(
      suppressWarnings(as.numeric(ncmas_year)),
      suppressWarnings(as.numeric(allocation_year)),
      get_end_year(source_period)
    )
  )

resource_envelope <- resource_envelope |>
  mutate(
    year = coalesce(
      suppressWarnings(as.numeric(ncmas_year)),
      suppressWarnings(as.numeric(allocation_year)),
      get_end_year(source_period)
```

```
    )
  )
```

5.3.1.5 Validate Demand and Allocation Integration

The integrated demand and allocation tables are checked to confirm that all yearly source workbooks have been retained and that the common analytical year is aligned consistently across the historical series.

```
context_metrics |>
  summarise(
    rows = n(),
    source_periods = n_distinct(source_period),
    source_files = n_distinct(source_file),
    years = n_distinct(year)
  )
```

```
# A tibble: 1 x 4
   rows source_periods source_files years
  <int>          <int>        <int> <int>
1   350             13           13    13
```

```
project_allocations |>
  summarise(
    rows = n(),
    source_periods = n_distinct(source_period),
    source_files = n_distinct(source_file),
    years = n_distinct(year)
  )
```

```
# A tibble: 1 x 4
   rows source_periods source_files years
  <int>          <int>        <int> <int>
1  5495             13           13    13
```

```
resource_envelope |>
  summarise(
    rows = n(),
    source_periods = n_distinct(source_period),
    source_files = n_distinct(source_file),
    years = n_distinct(year)
  )
```

```
# A tibble: 1 x 4
   rows source_periods source_files years
  <int>          <int>        <int> <int>
1    47             13           13    13
```

```
bind_rows(
  Context_Metrics = context_metrics |>
    distinct(source_period, year),
  Project_Allocations = project_allocations |>
    distinct(source_period, year),
  Resource_Envelope = resource_envelope |>
    distinct(source_period, year),
  .id = "table"
) |>
  count(
    table,
    source_period,
    name = "analytical_years"
  ) |>
  filter(analytical_years > 1)
```

```
# A tibble: 1 x 3
  table               source_period analytical_years
  <chr>               <chr>                    <int>
1 Project_Allocations 2014/15                      2
```

A further check is used to identify any source period that maps to more than one analytical year within an integrated table.

5.3.1.6 Integrate Field of Research Data

Field of Research composition data are combined across the yearly FoR workbooks while retaining the source filename and reporting period.

These records describe the disciplinary composition of HPC allocation and are integrated separately from the demand and allocation evidence because they have a different analytical structure and level of aggregation.

```
for_files <- list.files(
  for_path,
  pattern = "\\.xlsx$",
  full.names = TRUE
)
```

```
for_data <- map_dfr(
  for_files,
  function(file) {

    read_excel(
      file,
      sheet = "FoR_Composition",
      col_types = "text"
```

```
    ) |>
      mutate(
        source_file = basename(file)
      )
  }
)
```

```
for_data <- for_data |>
  mutate(
    year = get_end_year(source_period)
  )
```

5.3.1.7 Validate Field of Research Integration

The integrated Field of Research data are checked to confirm that all yearly FoR workbooks have been retained and that the common analytical year is aligned consistently with the demand and allocation series.

```
for_data |>
  summarise(
    rows = n(),
    source_periods = n_distinct(source_period),
    source_files = n_distinct(source_file),
    years = n_distinct(year)
  )
```

```
# A tibble: 1 x 4
   rows source_periods source_files years
  <int>          <int>        <int> <int>
1   133             13           13    13
```

5.3.1.8 Export Integrated Source Data

The validated integrated tables are exported as a consolidated workbook to provide a reproducible intermediate dataset between the original yearly source files and the annual modelling summary.

The workbook retains the different analytical components on separate worksheets so that differences in granularity and scope are preserved.

```
writexl::write_xlsx(
  list(
    Context_Metrics = context_metrics,
    Project_Allocations = project_allocations,
    Resource_Envelope = resource_envelope,
    FoR_Composition = for_data
  ),
  path = file.path(
```

```
    clean_path,
    "integrated_hpc_source_data.xlsx"
  )
)
```

5.4 Annual Demand and Allocation Summary Construction

The annual demand and allocation summary is constructed after integration of the detailed yearly source evidence.

The existing manually assembled summary is retained temporarily as a reference for reconciliation rather than treated as an independent source of data. Each annual requested and allocated compute value is checked against the integrated `Context_Metrics`, `Project_Allocations` and `Resource_Envelope` evidence.

Where a directly reported comparable annual value is available, that value is preferred. Where it is not available, a derived or proxy value may be constructed from the integrated evidence using an explicitly documented assumption.

5.4.1 Reconstruct Annual Demand and Allocation Summary

Annual demand and allocation measures are reconstructed from the integrated source tables before comparison with the previously assembled summary.

Context metrics are used to identify reported annual totals, ratios and derived demand information. Project-level allocations and resource envelopes are retained as supporting evidence and are aggregated only where their scope and units are sufficiently consistent.

```
context_candidates <- context_metrics |>
  mutate(
    value_num = suppressWarnings(as.numeric(value))
  ) |>
  filter(
    str_detect(
      metric,
      regex(
        "request|demand|allocat|fulfil|oversub|capacity|available",
        ignore_case = TRUE
      )
    )
  ) |>
  select(
    year,
```

```
    source_period,
    metric,
    value_num,
    unit,
    scope,
    source,
    notes
  ) |>
  arrange(year, metric)
```

```
project_candidates <- project_allocations |>
  mutate(
    allocated_compute_num =
      suppressWarnings(as.numeric(allocated_compute))
  ) |>
  filter(!is.na(allocated_compute_num)) |>
  group_by(
    year,
    source_period,
    unit,
    allocation_type
  ) |>
  summarise(
    allocated_total = sum(allocated_compute_num),
    records = n(),
    .groups = "drop"
  ) |>
  arrange(year, unit, allocation_type)
```

```
resource_candidates <- resource_envelope |>
  mutate(
    resource_available_num =
      suppressWarnings(as.numeric(resource_available))
  ) |>
  filter(!is.na(resource_available_num)) |>
  group_by(
    year,
    source_period,
    unit
  ) |>
  summarise(
    resource_available_total =
      sum(resource_available_num),
    records = n(),
    .groups = "drop"
```

```
  ) |>
  arrange(year, unit)
```

5.4.2 Construct Reconstructed Annual Summary

The annual series is reconstructed from the integrated evidence without using the previously assembled summary as an input.

Directly reported annual values are used where available. Derived values are calculated only where the required source values and relationships are contained in the integrated evidence. Where the integrated evidence is insufficient to reproduce an earlier summary value, the value is left missing for subsequent reconciliation rather than being inferred from the existing summary.

```
get_metric_value <- function(target_year, metric_name) {

  values <- context_candidates |>
    filter(
      year == target_year,
      metric == metric_name
    ) |>
    pull(value_num)

  if (length(values) != 1) {
    stop(
      paste(
        "Expected one value for",
        target_year,
        metric_name,
        "but found",
        length(values)
      )
    )
  }

  values
}
```

```
reconstructed_summary <- tibble(
  year = 2014:2026,
  source_period = paste0(
    year - 1,
    "/",
    str_sub(as.character(year), -2)
  )
) |>
```

```
mutate(

  requested_compute = case_when(

    year == 2014 ~
      get_metric_value(
        2014,
        "ncmas_total_ksu_request"
      ),

    year == 2015 ~
      get_metric_value(
        2015,
        "nci_ncmas_implied_demand_2015"
      ),

    year == 2016 ~
      get_metric_value(
        2016,
        "nci_ncmas_implied_requested_compute_2016"
      ),

    year == 2017 ~
      get_metric_value(
        2017,
        "pawsey_ncmas_requested_compute_2017"
      ),

    year == 2018 ~
      get_metric_value(
        2018,
        "national_implied_requested_compute_2018"
      ),

    # Current integrated evidence does not contain the
    # 220 × 3 national proxy used previously.
    year == 2019 ~ NA_real_,

    year == 2020 ~
      resource_candidates |>
        filter(
          year == 2020,
          unit == "kSU"
        ) |>
        pull(resource_available_total) |>
```

```
        first() * 2,

    year == 2021 ~
      get_metric_value(
        2021,
        "direct_ncmas_2021_total_allocation"
      ) *
      get_metric_value(
        2021,
        "ncmas_demand_supply_factor_high"
      ),

    year == 2022 ~
      get_metric_value(
        2022,
        "ncmas_allocated_compute_total"
      ) /
      (
        get_metric_value(
          2022,
          "ncmas_allocated_share_of_requested_pct"
        ) / 100
      ),

    year == 2023 ~
      get_metric_value(
        2023,
        "ncmas_requested_compute"
      ),

    # Retrospective 57% and 56% demand relationships are
    # not currently retained in the integrated source tables.
    year == 2024 ~ NA_real_,
    year == 2025 ~ NA_real_,

    year == 2026 ~
      get_metric_value(
        2026,
        "ncmas_requested_compute"
      )
  ),

  allocated_compute = case_when(

    # 83,505 kSU is not currently retained as an annual
```

```
    # metric in the integrated evidence.
    year == 2014 ~ NA_real_,

    year == 2015 ~
      get_metric_value(
        2015,
        "nci_ncmas_allocation_2014_15"
      ),

    year == 2016 ~
      get_metric_value(
        2016,
        "nci_ncmas_allocation_2015_16"
      ),

    year == 2017 ~
      get_metric_value(
        2017,
        "pawsey_ncmas_available_compute_2017"
      ),

    year == 2018 ~
      get_metric_value(
        2018,
        "national_allocated_compute_2018"
      ),

    # The 220m national capacity proxy is not currently
    # retained in the integrated evidence.
    year == 2019 ~ NA_real_,

    year == 2020 ~
      get_metric_value(
        2020,
        "nci_supplement_ncmas_allocation_total_verified"
      ),

    year == 2021 ~
      get_metric_value(
        2021,
        "direct_ncmas_2021_total_allocation"
      ),

    year == 2022 ~
      get_metric_value(
```

```
          2022,
          "ncmas_allocated_compute_total"
        ),

      year == 2023 ~
        get_metric_value(
          2023,
          "ncmas_allocated_compute_total"
        ),

      year == 2024 ~
        get_metric_value(
          2024,
          "ncmas_total_allocation"
        ),

      year == 2025 ~
        get_metric_value(
          2025,
          "ncmas_total_allocation"
        ),

      year == 2026 ~
        get_metric_value(2026, "ncmas_gadi_allocation") +
        get_metric_value(2026, "ncmas_setonix_cpu_allocation") +
        get_metric_value(2026, "ncmas_setonix_gpu_allocation")
    )
  )
```

```
reconstructed_summary
```

```
# A tibble: 13 x 4
    year source_period requested_compute allocated_compute
   <int> <chr>                     <dbl>             <dbl>
 1  2014 2013/14                131356                NA
 2  2015 2014/15                   202.            88731
 3  2016 2015/16                   225             97112
 4  2017 2016/17                   200               100
 5  2018 2017/18                   529.              222
 6  2019 2018/19                    NA                NA
 7  2020 2019/20                607980            255148
 8  2021 2020/21               1193811            397937
 9  2022 2021/22               1162230            581115
10  2023 2022/23               1333240            817956
11  2024 2023/24                    NA            812569
12  2025 2024/25                    NA            876200
```

13 2026 2025/26 2242817 872288

5.4.3 Compare Reconstructed and Existing Annual Summaries

The reconstructed annual series is compared with the previously assembled summary to identify values that are reproduced directly from the integrated evidence and values that require additional source evidence or documented derivation.

```
current_summary <- read_excel(
  file.path(clean_path, "demand_allocation_data.xlsx")
) |>
  mutate(
    year = get_end_year(source_period)
  )
```

```
summary_comparison <- current_summary |>
  select(
    year,
    source_period,
    old_requested = requested_compute,
    old_allocated = allocated_compute
  ) |>
  full_join(
    reconstructed_summary |>
      rename(
        reconstructed_requested = requested_compute,
        reconstructed_allocated = allocated_compute
      ),
    by = c("year", "source_period")
  ) |>
  mutate(
    requested_match = case_when(
      is.na(reconstructed_requested) ~ FALSE,
      TRUE ~ near(old_requested, reconstructed_requested)
    ),
    allocated_match = case_when(
      is.na(reconstructed_allocated) ~ FALSE,
      TRUE ~ near(old_allocated, reconstructed_allocated)
    )
  ) |>
  arrange(year)
```

```
summary_comparison |>
  select(
    year,
```

```
    source_period,
    old_requested,
    reconstructed_requested,
    requested_match,
    old_allocated,
    reconstructed_allocated,
    allocated_match
  )
```

```
# A tibble: 13 x 8
    year source_period old_requested reconstructed_requested requested_match
   <dbl> <chr>                 <dbl>                   <dbl> <lgl>
 1  2014 2013/14              131356                  131356  TRUE
 2  2015 2014/15                 202.                    202. TRUE
 3  2016 2015/16                 225                     225  TRUE
 4  2017 2016/17                 200                     200  TRUE
 5  2018 2017/18                 529.                    529. FALSE
 6  2019 2018/19                 660                      NA  FALSE
 7  2020 2019/20              607980                  607980  TRUE
 8  2021 2020/21             1193811                 1193811  TRUE
 9  2022 2021/22             1162230                 1162230  TRUE
10  2023 2022/23             1333240                 1333240  TRUE
11  2024 2023/24             1425560.                     NA  FALSE
12  2025 2024/25             1564643.                     NA  FALSE
13  2026 2025/26             2242817                 2242817  TRUE
# i 3 more variables: old_allocated <dbl>, reconstructed_allocated <dbl>,
#   allocated_match <lgl>
```

5.4.4 Resolve Unreconstructed Annual Values

The initial reconstruction identified several annual values that could not be reproduced from the integrated source tables because the supporting metric or derivation had not yet been retained explicitly in the integrated evidence.

These cases are reviewed against the original yearly source evidence before the final annual summary is constructed.

```
unreconstructed_values <- summary_comparison |>
  filter(
    is.na(reconstructed_requested) |
    is.na(reconstructed_allocated)
  ) |>
  transmute(
    year,
    source_period,
    missing_requested = is.na(reconstructed_requested),
    missing_allocated = is.na(reconstructed_allocated),
```

```
    old_requested,
    old_allocated
  )

unreconstructed_values
```

```
# A tibble: 4 x 6
   year source_period missing_requested missing_allocated old_requested
  <dbl> <chr>         <lgl>             <lgl>                      <dbl>
1  2014 2013/14       FALSE             TRUE                      131356
2  2019 2018/19       TRUE              TRUE                         660
3  2024 2023/24       TRUE              FALSE                    1425560.
4  2025 2024/25       TRUE              FALSE                    1564643.
# i 1 more variable: old_allocated <dbl>
```

The unresolved cases do not necessarily represent missing research activity. Rather, they identify years where no directly usable annual measure is available from the integrated source tables. These values therefore require documented derivation or proxy treatment based on the original source evidence.

5.4.5 Derive Missing Annual Measures

Where a directly reported annual demand or allocation measure is not available, a derived or proxy value is constructed from other published evidence retained in the integrated source data.

These derivations are applied only where the relationship used is explicitly supported by the source material. Derived and proxy values are retained separately from directly reported measures so that their status remains transparent.

```
reconstructed_summary <- reconstructed_summary |>
  mutate(
    requested_compute = case_when(
      year == 2019 ~ 220 * 3,
      year == 2024 ~ 812569 / 0.57,
      year == 2025 ~ 876200 / 0.56,
      TRUE ~ requested_compute
    ),
    allocated_compute = case_when(
      year == 2014 ~ 83505,
      year == 2019 ~ 220,
      TRUE ~ allocated_compute
    )
  )
```

```
reconstructed_summary <- reconstructed_summary |>
  mutate(
```

```
    requested_basis = case_when(
      year == 2014 ~ "Direct",
      year == 2015 ~ "Derived",
      year == 2016 ~ "Derived",
      year == 2017 ~ "Proxy",
      year == 2018 ~ "Derived",
      year == 2019 ~ "Derived proxy",
      year == 2020 ~ "Derived proxy",
      year == 2021 ~ "Derived proxy",
      year == 2022 ~ "Derived proxy",
      year == 2023 ~ "Direct",
      year == 2024 ~ "Derived proxy",
      year == 2025 ~ "Derived proxy",
      year == 2026 ~ "Direct"
    ),

    allocated_basis = case_when(
      year == 2014 ~ "Direct",
      year == 2015 ~ "Direct",
      year == 2016 ~ "Direct",
      year == 2017 ~ "Capacity proxy",
      year == 2018 ~ "Direct",
      year == 2019 ~ "Capacity proxy",
      year == 2020 ~ "Direct",
      year == 2021 ~ "Direct",
      year == 2022 ~ "Direct",
      year == 2023 ~ "Direct",
      year == 2024 ~ "Direct",
      year == 2025 ~ "Direct",
      year == 2026 ~ "Direct comparable subset"
    ),

    derivation_note = case_when(
      year == 2015 ~
        "Requested compute derived from reported allocation and demand relationship.",

      year == 2016 ~
        "Requested compute derived from reported one-third fulfilment relationship.",

      year == 2017 ~
        "Requested compute retained as a lower-bound proxy; available capacity used as the a

      year == 2018 ~
        "Requested compute derived from 222 million hours and a 42% fulfilment relationship.
```

```
      year == 2019 ~
        "Requested compute derived as 220 × 3; available capacity used as the allocation pro

      year == 2020 ~
        "Requested compute derived from the resource envelope and reported demand-supply rel

      year == 2021 ~
        "Requested compute derived using the approximate three-times demand-supply relations

      year == 2022 ~
        "Requested compute derived from allocated compute and the reported 50% fulfilment re

      year == 2024 ~
        "Requested compute derived from allocated compute and the retrospective 57% fulfilme

      year == 2025 ~
        "Requested compute derived from allocated compute and the retrospective 56% fulfilme

      year == 2026 ~
        "Setonix-Q excluded from allocation to match the scope of the published requested-co

      TRUE ~ NA_character_
    )
  )
```

```
reconstructed_summary
```

```
# A tibble: 13 x 7
    year source_period requested_compute allocated_compute requested_basis
   <int> <chr>                    <dbl>             <dbl> <chr>
 1  2014 2013/14                131356              83505 Direct
 2  2015 2014/15                   202.             88731 Derived
 3  2016 2015/16                   225              97112 Derived
 4  2017 2016/17                   200                100 Proxy
 5  2018 2017/18                   529.               222 Derived
 6  2019 2018/19                   660                220 Derived proxy
 7  2020 2019/20                607980             255148 Derived proxy
 8  2021 2020/21               1193811             397937 Derived proxy
 9  2022 2021/22               1162230             581115 Derived proxy
10  2023 2022/23               1333240             817956 Direct
11  2024 2023/24               1425560.            812569 Derived proxy
12  2025 2024/25               1564643.            876200 Derived proxy
13  2026 2025/26               2242817             872288 Direct
# i 2 more variables: allocated_basis <chr>, derivation_note <chr>
```

5.4.6 Final Reconciliation Check

After the missing annual measures are reconstructed and their basis is documented, the annual series is compared again with the previously assembled summary.

This final check identifies whether the reconstructed values reproduce the existing annual summary and highlights any remaining differences due to rounding, scope or methodological treatment.

```
final_summary_comparison <- current_summary |>
  select(
    year,
    source_period,
    old_requested = requested_compute,
    old_allocated = allocated_compute
  ) |>
  full_join(
    reconstructed_summary |>
      select(
        year,
        source_period,
        reconstructed_requested = requested_compute,
        reconstructed_allocated = allocated_compute,
        requested_basis,
        allocated_basis,
        derivation_note
      ),
    by = c("year", "source_period")
  ) |>
  mutate(
    requested_difference =
      reconstructed_requested - old_requested,

    allocated_difference =
      reconstructed_allocated - old_allocated,

    requested_match =
      near(
        old_requested,
        reconstructed_requested,
        tol = 1e-3
      ),

    allocated_match =
      near(
        old_allocated,
```

```
        reconstructed_allocated,
        tol = 1e-3
      )
  ) |>
  arrange(year)
```

```
final_summary_comparison |>
  select(
    year,
    source_period,
    old_requested,
    reconstructed_requested,
    requested_match,
    old_allocated,
    reconstructed_allocated,
    allocated_match
  )
```

```
# A tibble: 13 x 8
    year source_period old_requested reconstructed_requested requested_match
   <dbl> <chr>                 <dbl>                   <dbl> <lgl>
 1  2014 2013/14              131356                  131356  TRUE
 2  2015 2014/15                 202.                    202. TRUE
 3  2016 2015/16                 225                     225  TRUE
 4  2017 2016/17                 200                     200  TRUE
 5  2018 2017/18                 529.                    529. TRUE
 6  2019 2018/19                 660                     660  TRUE
 7  2020 2019/20              607980                  607980  TRUE
 8  2021 2020/21             1193811                 1193811  TRUE
 9  2022 2021/22             1162230                 1162230  TRUE
10  2023 2022/23             1333240                 1333240  TRUE
11  2024 2023/24             1425560.                1425560. TRUE
12  2025 2024/25             1564643.                1564643. TRUE
13  2026 2025/26             2242817                 2242817  TRUE
# i 3 more variables: old_allocated <dbl>, reconstructed_allocated <dbl>,
#   allocated_match <lgl>
```

```
final_summary_comparison |>
  summarise(
    years = n(),
    requested_matches = sum(requested_match),
    allocated_matches = sum(allocated_match),
    requested_mismatches = sum(!requested_match),
    allocated_mismatches = sum(!allocated_match)
  )
```

```
# A tibble: 1 x 5
```

```
  years requested_matches allocated_matches requested_mismatches
  <int>             <int>             <int>                 <int>
1    13                13                13                     0
# i 1 more variable: allocated_mismatches <int>
```

5.4.7 Export Reconciled Annual Demand and Allocation Summary

Following reconciliation, the reconstructed annual series is retained as the reproducible demand and allocation summary for subsequent standardisation, conversion and modelling.

The original reporting unit is preserved alongside each annual observation because the historical series is not yet expressed on a common compute scale. In years where requested and allocated compute were reported in different units, the mixed-unit description is retained explicitly.

The export also preserves whether each measure is directly reported, derived or proxy-based so that these distinctions remain available during later conversion and modelling.

```
final_demand_allocation_summary <- reconstructed_summary |>
  left_join(
    current_summary |>
      select(
        year,
        source_period,
        unit,
        requested_source,
        allocated_source,
        source_url
      ),
    by = c("year", "source_period")
  ) |>
  select(
    year,
    source_period,
    requested_compute,
    allocated_compute,
    unit,
    requested_basis,
    allocated_basis,
    requested_source,
    allocated_source,
    derivation_note,
    source_url
  )
```

```
final_demand_allocation_summary
```

```
# A tibble: 13 x 11
    year source_period requested_compute allocated_compute unit  requested_basis
   <dbl> <chr>                     <dbl>             <dbl> <chr> <chr>
 1  2014 2013/14                  131356             83505 kSU   Direct
 2  2015 2014/15                     202.            88731 mixe~ Derived
 3  2016 2015/16                     225             97112 mixe~ Derived
 4  2017 2016/17                     200               100 mill~ Proxy
 5  2018 2017/18                     529.              222 mill~ Derived
 6  2019 2018/19                     660               220 mill~ Derived proxy
 7  2020 2019/20                  607980            255148 kSU   Derived proxy
 8  2021 2020/21                 1193811            397937 kSU   Derived proxy
 9  2022 2021/22                 1162230            581115 kSU   Derived proxy
10  2023 2022/23                 1333240            817956 kSU   Direct
11  2024 2023/24                 1425560.           812569 kSU   Derived proxy
12  2025 2024/25                 1564643.           876200 kSU   Derived proxy
13  2026 2025/26                 2242817            872288 kSU   Direct
# i 5 more variables: allocated_basis <chr>, requested_source <chr>,
#   allocated_source <chr>, derivation_note <chr>, source_url <chr>
```

```
writexl::write_xlsx(
  final_demand_allocation_summary,
  path = file.path(
    clean_path,
    "reconciled_demand_allocation_summary.xlsx"
  )
)
```

5.5 Field of Research Summary Construction

The integrated Field of Research records are next assembled into a consistent annual summary.

Unlike the national demand and allocation series, historical FoR evidence varies in classification, reporting scope and completeness. The purpose of this stage is therefore to preserve the reported disciplinary composition while identifying differences in coverage and evidence quality across years.

FoR percentages are treated primarily as allocation composition rather than directly observed demand composition. Where these shares are later applied to annual requested compute, the resulting values are treated as analytical demand proxies rather than observed FoR-specific demand.

```
for_annual_check <- for_data |>
  mutate(
```

```
    composition_pct_num =
      suppressWarnings(as.numeric(composition_pct))
  ) |>
  group_by(
    year,
    source_period
  ) |>
  summarise(
    records = n(),
    composition_total =
      sum(composition_pct_num, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(year)

for_annual_check
```

```
# A tibble: 13 x 4
    year source_period records composition_total
   <dbl> <chr>           <int>             <dbl>
 1  2014 2013/14             7               100
 2  2015 2014/15             5               100
 3  2016 2015/16             7               100
 4  2017 2016/17            17               100
 5  2018 2017/18             8               100
 6  2019 2018/19             9               100
 7  2020 2019/20             8               100
 8  2021 2020/21             9               100
 9  2022 2021/22            13               100.
10  2023 2022/23            12               100.0
11  2024 2023/24            17               100
12  2025 2024/25            12               100.0
13  2026 2025/26             9               100.0
```

```
for_quality_summary <- for_data |>
  mutate(
    composition_pct_num =
      suppressWarnings(as.numeric(composition_pct))
  ) |>
  group_by(
    year,
    source_period,
    evidence_quality,
    scope,
    value_basis
  ) |>
```

```
  summarise(
    records = n(),
    composition_total =
      sum(composition_pct_num, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(year)

for_quality_summary
```

```
# A tibble: 19 x 7
    year source_period evidence_quality                scope value_basis records
   <dbl> <chr>         <chr>                           <chr> <chr>         <int>
 1  2014 2013/14       Derived residual from publishe~ NCI ~ Residual o~       1
 2  2014 2013/14       Direct percentage / scope unce~ NCI ~ Allocation~       6
 3  2015 2014/15       Direct percentage / NCMAS scop~ NCMA~ Allocation~       5
 4  2016 2015/16       Derived residual from publishe~ Meri~ Residual o~       1
 5  2016 2015/16       Direct percentage / scheme sco~ Meri~ Allocation~       6
 6  2017 2016/17       Derived residual from publishe~ Paws~ Residual o~       1
 7  2017 2016/17       Direct percentage / Pawsey all~ Paws~ Usage comp~      16
 8  2018 2017/18       Estimated - project-title clas~ NCI ~ Estimated ~       8
 9  2019 2018/19       Derived residual from publishe~ NCI ~ Residual o~       1
10  2019 2018/19       Estimated / incomplete extract~ NCI ~ Estimated ~       8
11  2020 2019/20       Estimated from near-complete N~ NCI ~ Estimated ~       8
12  2021 2020/21       Derived residual from publishe~ Nati~ Residual o~       1
13  2021 2020/21       Estimated from complete direct~ Nati~ Allocation~       8
14  2022 2021/22       Strong composition evidence: s~ Nati~ Allocation~      13
15  2023 2022/23       Strong composition evidence     Nati~ Allocation~      12
16  2024 2023/24       Source-preserving residual      Nati~ Allocation~       1
17  2024 2023/24       Strong composition evidence     Nati~ Allocation~      16
18  2025 2024/25       Strong composition evidence     Nati~ Allocation~      12
19  2026 2025/26       Strong composition evidence     Nati~ Allocation~       9
# i 1 more variable: composition_total <dbl>
```

5.5.1 Assess Historical FoR Comparability

The yearly FoR compositions generally sum to approximately 100%, but completeness alone does not imply that all years are directly comparable.

Historical observations differ in reporting scope, classification method and evidence quality. Some years are based on direct NCMAS allocation composition, while others rely on estimated project classification, residual categories, or broader facility-level usage.

For this reason, the FoR series is classified by analytical use rather than treated as a uniformly consistent time series. The strongest and most comparable

national NCMAS composition evidence is available from 2021 onward. Earlier observations are retained primarily as historical context, with 2017 treated separately because it represents Pawsey usage across all allocation schemes rather than national NCMAS allocation composition.

```
for_year_quality <- for_annual_check |>
  mutate(
    analytical_use = case_when(
      year == 2017 ~
        "Context only",

      year %in% c(
        2014,
        2016,
        2018,
        2019,
        2020
      ) ~
        "Historical / lower confidence",

      year == 2015 ~
        "Historical / moderate confidence",

      year >= 2021 ~
        "Main FoR trend"
    )
  )

for_year_quality
```

```
# A tibble: 13 x 5
    year source_period records composition_total analytical_use
   <dbl> <chr>           <int>             <dbl> <chr>
 1  2014 2013/14             7             100   Historical / lower confidence
 2  2015 2014/15             5             100   Historical / moderate confiden~
 3  2016 2015/16             7             100   Historical / lower confidence
 4  2017 2016/17            17             100   Context only
 5  2018 2017/18             8             100   Historical / lower confidence
 6  2019 2018/19             9             100   Historical / lower confidence
 7  2020 2019/20             8             100   Historical / lower confidence
 8  2021 2020/21             9             100   Main FoR trend
 9  2022 2021/22            13             100.  Main FoR trend
10  2023 2022/23            12             100.0 Main FoR trend
11  2024 2023/24            17             100   Main FoR trend
12  2025 2024/25            12             100.0 Main FoR trend
13  2026 2025/26             9             100.0 Main FoR trend
```

5.5.2 Construct FoR-Level Compute Measures

The annual FoR composition is linked to the reconciled national demand and allocation series using analytical year and source period.

Where FoR scope is sufficiently compatible with the annual national series, allocation shares are applied to annual compute totals to construct FoR-level estimates.

Years classified as contextual only are retained for composition analysis but are not used to distribute national annual compute totals because their reporting scope is not sufficiently comparable. The same shares are also applied to annual requested compute to construct an indicative FoR-level demand proxy.

The resulting demand measure is not treated as directly observed disciplinary demand because the available FoR percentages primarily describe allocation composition rather than request composition.

```
for_summary <- for_data |>
  mutate(
    composition_pct_num =
      suppressWarnings(as.numeric(composition_pct))
  ) |>
  left_join(
    for_year_quality |>
      select(
        year,
        source_period,
        analytical_use
      ),
    by = c("year", "source_period")
  )
```

```
for_summary <- for_summary |>
  left_join(
    final_demand_allocation_summary |>
      select(
        year,
        source_period,
        requested_compute,
        allocated_compute,
        unit,
        requested_basis,
        allocated_basis
      ),
    by = c("year", "source_period")
  )
```

```r
for_summary <- for_summary |>
  mutate(
    allocation_share_for =
      composition_pct_num / 100,

    scope_alignment = case_when(
      year == 2017 ~
        "Not comparable",

      year == 2026 ~
        "Review required: FoR includes Setonix-Q while annual comparable allocation excludes

      analytical_use == "Main FoR trend" ~
        "Broadly comparable",

      TRUE ~
        "Historical / approximate"
    ),

    estimated_allocated_compute_by_for = case_when(
      analytical_use == "Context only" ~ NA_real_,
      TRUE ~ allocated_compute * allocation_share_for
    ),

    estimated_demand_by_for_proxy = case_when(
      analytical_use == "Context only" ~ NA_real_,
      TRUE ~ requested_compute * allocation_share_for
    )
  )
```

```r
for_validation <- for_summary |>
  group_by(
    year,
    source_period,
    analytical_use
  ) |>
  summarise(
    composition_total =
      sum(composition_pct_num, na.rm = TRUE),

    estimated_allocated_for_total =
      if (
        all(is.na(estimated_allocated_compute_by_for))
      ) {
        NA_real_
```

```
      } else {
        sum(
          estimated_allocated_compute_by_for,
          na.rm = TRUE
        )
      },

    demand_proxy_total =
      if (
        all(is.na(estimated_demand_by_for_proxy))
      ) {
        NA_real_
      } else {
        sum(
          estimated_demand_by_for_proxy,
          na.rm = TRUE
        )
      },

    annual_allocated =
      first(allocated_compute),

    annual_requested =
      first(requested_compute),

    .groups = "drop"
  ) |>
  arrange(year)

for_validation
```

```
# A tibble: 13 x 8
    year source_period analytical_use   composition_total estimated_allocated_~1
   <dbl> <chr>         <chr>                        <dbl>                   <dbl>
 1  2014 2013/14       Historical / lo~               100                   83505
 2  2015 2014/15       Historical / mo~               100                   88731
 3  2016 2015/16       Historical / lo~               100                   97112
 4  2017 2016/17       Context only                   100                      NA
 5  2018 2017/18       Historical / lo~               100                     222
 6  2019 2018/19       Historical / lo~               100                     220
 7  2020 2019/20       Historical / lo~               100                  255148
 8  2021 2020/21       Main FoR trend                 100                  397937
 9  2022 2021/22       Main FoR trend                 100.                 581115.
10  2023 2022/23       Main FoR trend                 100.0                817956.
11  2024 2023/24       Main FoR trend                 100                  812569
```

```
12  2025 2024/25       Main FoR trend               100.0                 876200.
13  2026 2025/26       Main FoR trend               100.0                 872288.
# i abbreviated name: 1: estimated_allocated_for_total
# i 3 more variables: demand_proxy_total <dbl>, annual_allocated <dbl>,
#   annual_requested <dbl>
```

The validation confirms that the FoR composition shares sum to 100% for each analytical year. As a result, the sum of FoR-level allocated compute reproduces the corresponding annual allocated-compute total, while the sum of the FoR-level demand proxies reproduces the annual requested-compute total.

This confirms that the FoR allocation shares have been applied consistently to the reconciled national series. The demand values remain analytical proxies rather than directly observed FoR-specific requests.

5.5.3 Export Reconciled Field of Research Summary

The validated FoR-level measures are retained as the integrated disciplinary summary for subsequent standardisation, analysis and modelling.

The exported table preserves the original FoR classification information, allocation composition, analytical-use classification and evidence-quality metadata. FoR-level requested compute remains explicitly identified as a proxy derived from allocation composition.

```
final_for_summary <- for_summary |>
  select(
    year,
    source_period,

    research_field_reported,
    for_2008_division_code,
    for_2020_division_code,
    for_standard_2020,

    composition_pct = composition_pct_num,
    allocation_share_for,

    requested_compute_annual = requested_compute,
    allocated_compute_annual = allocated_compute,
    annual_compute_unit = unit,

    estimated_demand_by_for_proxy,
    estimated_allocated_compute_by_for,

    requested_basis,
    allocated_basis,
```

```
    analytical_use,
    scope_alignment,
    evidence_quality,
    scope,
    value_basis,
    classification_status,
    standardisation_note,
    notes,
    source,
    source_url
  ) |>
  arrange(
    year,
    desc(composition_pct)
  )
```

```
final_for_summary |>
  summarise(
    rows = n(),
    years = n_distinct(year),
    source_periods = n_distinct(source_period),
    missing_composition = sum(is.na(composition_pct)),
    missing_for_name = sum(
      is.na(for_standard_2020) |
        str_trim(for_standard_2020) == ""
    )
  )
```

```
# A tibble: 1 x 5
   rows years source_periods missing_composition missing_for_name
  <int> <int>          <int>               <int>            <int>
1   133    13             13                   0                0
```

```
writexl::write_xlsx(
  final_for_summary,
  path = file.path(
    clean_path,
    "reconciled_field_of_research_summary.xlsx"
  )
)
```

Chapter 6

Data Standardisation

6.1 Purpose

The purpose of this step is to standardise the integrated historical High-Performance Computing (HPC) data to support consistent analysis across reporting years.

This stage uses the reconciled annual demand and allocation summary and the Field of Research (FoR) summary produced during Data Integration.

Standardisation focuses on aligning reporting periods, variable names, units, research field classifications, and data quality labels where necessary. Original values, source information, and derivation assumptions are retained to maintain traceability.

Numerical demand and allocation values are not converted at this stage. Compute-unit conversion is addressed separately in the Data Conversion chapter.

6.2 Reporting Period

Reporting periods are standardised using the ending year of each annual reporting period. For example, 2013/14 is represented as 2014, and 2025/26 as 2026.

The analysis covers 13 annual observations from 2014 to 2026. The year column is retained as the analytical year, while source_period preserves the original reporting period.

Both variables are carried forward from Data Integration. This step validates their consistency without modifying the existing year assignments.

```r
library(dplyr)
library(readxl)

annual_data <- read_excel(
  "assets/data/data clean/reconciled_demand_allocation_summary.xlsx"
)

year_check <- annual_data |>
  mutate(
    expected_year = as.integer(
      sub("^.*[/\\-]", "", source_period)
    ) + 2000L
  ) |>
  filter(is.na(year) | year != expected_year)

year_check
```

```
# A tibble: 0 x 12
# i 12 variables: year <dbl>, source_period <chr>, requested_compute <dbl>,
#   allocated_compute <dbl>, unit <chr>, requested_basis <chr>,
#   allocated_basis <chr>, requested_source <chr>, allocated_source <chr>,
#   derivation_note <chr>, source_url <chr>, expected_year <int>
```

```r
stopifnot(nrow(year_check) == 0)
```

The validation identified no inconsistencies between the analytical years and their corresponding reporting periods.

6.3 Institution Names

Institution names may differ across historical sources because of abbreviations, naming conventions, or organisational changes.

The reconciled annual demand and allocation summary and Field of Research (FoR) summary do not contain institution-level observations. Therefore, no institution-name mapping is applied to these datasets.

Institution-level standardisation may be undertaken separately if institutional allocation data are included in subsequent analysis.

6.4 Field of Research Classification

Historical Field of Research (FoR) data use different classification systems, including ANZSRC 2008 and ANZSRC 2020.

The reconciled FoR dataset from Data Integration retains the originally reported

research fields and classification codes, alongside a standardised FoR category for longitudinal analysis.

Existing mappings are retained, including combined historical categories where a direct one-to-one mapping is not supported. Classification status and standardisation notes document estimated classifications, unresolved mappings, and differences between reporting systems.

This step validates the existing classifications without changing allocation shares, demand estimates, or other numerical values.

```
for_data <- read_excel(
  "assets/data/data clean/reconciled_field_of_research_summary.xlsx"
)

# Review existing classifications
for_classification_check <- for_data |>
  count(
    classification_status,
    standardisation_note,
    name = "records"
  ) |>
  arrange(desc(records))

for_classification_check
```

```
# A tibble: 32 x 3
   classification_status                              standardisation_note records
   <chr>                                              <chr>                  <int>
 1 Derived from source-reported 6-digit FoR percen~ Mapped from reporte~      41
 2 ANZSRC 2020 source codes; allocation-weighted f~ FoR 2020 division r~      21
 3 As reported; FoR 2008/2020 alignment pending     Mapped from FoR 200~      11
 4 As reported; Pawsey area-of-science category; F~ Mapped from FoR 200~      10
 5 Added residual to complete published compositio~ No FoR code               5
 6 Estimated broad research-field classification f~ Mapped from FoR 200~       5
 7 Estimated from 148 supplied positive-NCMAS NCI ~ Mapped from FoR 200~       4
 8 Estimated from direct NCMAS 2021 project titles~ Mapped from FoR 200~       4
 9 Estimated - project-title classification; alloc~ Mapped from FoR 200~       4
10 As reported; FoR 2008/2020 alignment pending     Historical combined~       3
# i 22 more rows
```

```
# Identify records requiring mapping review
for_mapping_review <- for_data |>
  filter(
    is.na(for_standard_2020) |
      grepl(
        "pending|estimated|cannot split",
        paste(classification_status,
```

```
              standardisation_note),
        ignore.case = TRUE
      )
  ) |>
  select(
    year,
    research_field_reported,
    for_standard_2020,
    classification_status,
    standardisation_note
  )

for_mapping_review
```

```
# A tibble: 64 x 5
    year research_field_reported        for_standard_2020  classification_status
   <dbl> <chr>                         <chr>              <chr>
 1  2014 Earth & Environmental Sciences Earth sciences + ~ As reported; FoR 200~
 2  2014 Physical Sciences              Physical sciences  As reported; FoR 200~
 3  2014 Chemical Sciences              Chemical sciences  As reported; FoR 200~
 4  2014 Engineering & Technology       Engineering + Tec~ As reported; FoR 200~
 5  2014 Biological Sciences            Biological scienc~ As reported; FoR 200~
 6  2014 Mathematical Sciences          Mathematical scie~ As reported; FoR 200~
 7  2015 Earth & Environmental Sciences Earth sciences + ~ As reported; FoR 200~
 8  2015 Physical Sciences              Physical sciences  As reported; FoR 200~
 9  2015 Chemical Sciences              Chemical sciences  As reported; FoR 200~
10  2015 Engineering & Technology       Engineering + Tec~ As reported; FoR 200~
# i 54 more rows
# i 1 more variable: standardisation_note <chr>
```

6.4.1 Historical FoR Reporting Groups

For visualisation, the FoR data are divided into two periods: 2014–2020 and 2021–2026.

For 2014 - 2020, originally reported research fields are grouped into consistent historical reporting categories. In particular, the separately reported Geosciences, Earth Sciences, and Environmental Sciences categories are combined into Earth & Environmental Sciences. Radioastronomy is combined with Physical Sciences, while Engineering and Technology are combined into Engineering & Technology.

For 2021 - 2026, the existing standardised ANZSRC 2020 FoR categories are retained.

These groupings are used for reporting and visualisation only. Original research field names, allocation shares, estimated compute values, and source information remain unchanged. Differences in data scope, particularly Pawsey-wide

supercomputer usage in 2016/17, are retained as comparability limitations.

```
# Create FoR reporting categories without modifying source values
for_data <- for_data |>
  mutate(
    for_analysis_group = case_when(
      year <= 2020 &
        research_field_reported %in% c(
          "Geosciences",
          "Earth Sciences (exc. Geosciences)",
          "Environmental Sciences",
          "Earth & Environmental Sciences"
        ) ~ "Earth & Environmental Sciences",

      year <= 2020 &
        research_field_reported %in% c(
          "Radioastronomy",
          "Physical Sciences (exc. radio astronomy)",
          "Physical Sciences"
        ) ~ "Physical Sciences",

      year <= 2020 &
        research_field_reported %in% c(
          "Engineering",
          "Technology",
          "Engineering & Technology"
        ) ~ "Engineering & Technology",

      year <= 2020 &
        research_field_reported %in% c(
          "Other / Unreported",
          "Other / Unclassified"
        ) ~ "Other / Unclassified",

      year <= 2020 ~ research_field_reported,

      TRUE ~ for_standard_2020
    )
  )

# Review the harmonised historical categories
historical_for_check <- for_data |>
  filter(year <= 2020) |>
  group_by(year, for_analysis_group) |>
  summarise(
    allocation_share_pct = sum(
```

```
      allocation_share_for, na.rm = TRUE
    ) * 100,
    .groups = "drop"
  ) |>
  arrange(year, desc(allocation_share_pct))

historical_for_check
```

```
# A tibble: 56 x 3
    year for_analysis_group             allocation_share_pct
   <dbl> <chr>                                         <dbl>
 1  2014 Earth & Environmental Sciences                   42
 2  2014 Physical Sciences                                22
 3  2014 Chemical Sciences                                17
 4  2014 Engineering & Technology                         11
 5  2014 Biological Sciences                               4
 6  2014 Other / Unclassified                              3
 7  2014 Mathematical Sciences                             1
 8  2015 Earth & Environmental Sciences                   39
 9  2015 Physical Sciences                                23
10  2015 Chemical Sciences                                21
# i 46 more rows
```

```
# Confirm the new grouping does not alter annual shares
share_validation <- for_data |>
  group_by(year) |>
  summarise(
    original_share = sum(allocation_share_for, na.rm = TRUE),
    .groups = "drop"
  ) |>
  left_join(
    for_data |>
      group_by(year, for_analysis_group) |>
      summarise(
        grouped_share = sum(allocation_share_for, na.rm = TRUE),
        .groups = "drop"
      ) |>
      group_by(year) |>
      summarise(
        grouped_share = sum(grouped_share),
        .groups = "drop"
      ),
    by = "year"
  ) |>
  mutate(
    difference = original_share - grouped_share
```

```
  )

share_validation
```

```
# A tibble: 13 x 4
    year original_share grouped_share difference
   <dbl>          <dbl>         <dbl>      <dbl>
 1  2014          1             1              0
 2  2015          1             1              0
 3  2016          1             1              0
 4  2017          1             1              0
 5  2018          1             1              0
 6  2019          1             1              0
 7  2020          1             1              0
 8  2021          1             1              0
 9  2022          1.00          1.00           0
10  2023          1.000         1.000          0
11  2024          1             1              0
12  2025          1.000         1.000          0
13  2026          1.000         1.000          0
```

```
stopifnot(
  all(abs(share_validation$difference) < 1e-10)
)

# Confirm every record has a reporting category
missing_groups <- for_data |>
  filter(is.na(for_analysis_group) | for_analysis_group == "")

missing_groups
```

```
# A tibble: 0 x 26
# i 26 variables: year <dbl>, source_period <chr>,
#   research_field_reported <chr>, for_2008_division_code <chr>,
#   for_2020_division_code <chr>, for_standard_2020 <chr>,
#   composition_pct <dbl>, allocation_share_for <dbl>,
#   requested_compute_annual <dbl>, allocated_compute_annual <dbl>,
#   annual_compute_unit <chr>, estimated_demand_by_for_proxy <dbl>,
#   estimated_allocated_compute_by_for <dbl>, requested_basis <chr>, ...
```

```
stopifnot(nrow(missing_groups) == 0)
```

6.5 HPC Facilities and Systems

Historical HPC demand and allocation data originate from Australia's national computing facilities, including the National Computational Infrastructure (NCI)

and Pawsey Supercomputing Research Centre.

The reconciled annual summary retains facility and system information within the original source descriptions. These include NCI allocation records, Pawsey historical reporting, and more recent NCMAS outcomes identifying Gadi and Setonix CPU and GPU resources separately.

Facility and system information is therefore available for some reporting periods, although it is not consistently represented in dedicated structured variables across the integrated annual dataset.

At this stage, the existing facility and system descriptions are retained without modifying the original records. Where system-level information is available, it will inform resource classification and conversion assumptions in the Data Conversion chapter.

Differences between NCI and Pawsey accounting conventions, system configurations, and CPU/GPU resource measures are preserved as comparability considerations. No assumption is made that allocations from different systems are directly equivalent.

6.6 Resource Type

Historical HPC demand and allocation records include different computing resources, primarily CPU and GPU systems. The availability of resource-level information varies across reporting periods.

Earlier reporting years generally provide aggregate allocation or demand figures without a consistent CPU/GPU breakdown. Some historical systems, including Raijin and Magnus, are identified in the source records, but their system-level allocation shares are not always available.

More detailed resource-level information is available for later years. The 2020/21 allocation records identify Magnus, FlashLite, MASSIVE, and Gadi, while the 2021/22 records distinguish MASSIVE, Gadi, Setonix CPU, and Setonix GPU. From 2022/23 onwards, several reporting years provide separate Gadi CPU, Setonix CPU, and Setonix GPU allocations.

However, the availability of resource-level demand data is more limited. In particular, the 2023/24 and 2024/25 demand estimates are derived from aggregate allocation and fulfilment rates, without a directly reported CPU/GPU demand breakdown.

For standardisation, resource types are classified as CPU, GPU, or Unspecified only where supported by the original source information. Records without sufficient resource-level detail retain an unspecified classification rather than being assigned an estimated CPU/GPU split.

No numerical allocation or demand values are redistributed between resource

types at this stage. Resource-specific conversion factors and comparability adjustments are addressed in the Data Conversion chapter.

6.7 Units

The reconciled annual demand and allocation summary contains different computational accounting units across reporting periods.

The 2013/14 records are expressed in kSU. The 2014/15 and 2015/16 records contain mixed units, with demand expressed in million hours and allocation expressed in kSU. The 2016/17 records use million core-hours, while 2017/18 uses million hours and 2018/19 uses million CPU-hours.

From 2019/20 to 2025/26, annual demand and allocation values are recorded in kSU. However, the use of the same unit label does not necessarily establish equivalence between different HPC systems or CPU and GPU resources.

At the standardisation stage, original unit labels are retained without numerical conversion. Mixed-unit records are explicitly identified to prevent inappropriate direct comparisons.

The Data Conversion chapter will assess system-specific conversion factors, distinguish CPU and GPU accounting where supported, and document any assumptions required to establish comparable analytical measures.

6.8 Data Quality and Comparability

The integrated historical HPC dataset combines directly reported demand and allocation figures with values derived from reported demand-to-supply relationships, fulfilment rates, and available computing capacity. Consequently, the quality and comparability of observations vary across reporting years.

6.8.1 Data Provenance and Estimation

Directly reported observations are retained as published in the original sources. Where demand figures are unavailable, existing estimates derived during Data Integration are preserved together with their calculation assumptions and source references.

For example, the 2014/15 and 2015/16 demand estimates are derived from reported demand-to-supply or fulfilment relationships. The 2017/18, 2020/21, 2023/24, and 2024/25 demand estimates similarly rely on reported allocation totals and demand-to-supply or fulfilment information.

The 2016/17 and 2018/19 records include capacity-based information or conservative demand proxies. These are distinguished from directly reported allocation and demand figures.

No demand estimates or allocation values are recalculated during Data Standardisation.

6.8.2 Comparability Across Reporting Years

Historical observations differ in their reporting scope, computational accounting units, and availability of system-level information.

In particular, the 2016/17 data include Pawsey-wide supercomputer usage and capacity information, which are not directly equivalent to NCMAS allocation outcomes in other years.

Differences between CPU and GPU resources, system-specific accounting conventions, and incomplete resource breakdowns also limit direct comparisons across reporting periods.

These limitations are retained in the analysis rather than resolved through unsupported assumptions.

6.8.3 Treatment for Subsequent Analysis

The standardisation process preserves original numerical values, source references, estimation assumptions, and existing data quality information.

Observations with different reporting scopes, derived demand values, or uncertain conversion assumptions will be explicitly considered during Data Conversion, exploratory analysis, and forecasting model evaluation.

Where appropriate, sensitivity analysis will be used to assess how historical proxies and comparability limitations affect the forecasting results.

6.9 Output

The Data Standardisation stage prepares the reconciled historical HPC datasets for subsequent conversion, analysis, and forecasting.

Two datasets are carried forward:

- **Annual Demand and Allocation Summary:** Retains the 13 annual observations from 2014 to 2026, including original demand and allocation values, reporting periods, accounting units, source references, and existing estimation information. No numerical conversions are performed at this stage.
- **Standardised Field of Research Summary:** Retains the original FoR records and classification information while adding `for_analysis_group` for consistent historical reporting and visualisation.

The standardisation process preserves original numerical values, research field labels, source references, and estimation assumptions. Historical FoR categories

are harmonised for reporting purposes without redistributing allocation shares.

Validation checks confirm reporting-period consistency, preservation of annual FoR allocation-share totals, and the absence of missing FoR reporting groups.

The resulting datasets provide the input for Data Conversion, where computational accounting units and system-specific conversion assumptions will be evaluated before numerical comparisons and forecasting are undertaken.

```
# Create a separate folder for standardisation outputs
output_dir <- "assets/data/data standardised"

dir.create(
  output_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

# Save the annual summary without changing its values
saveRDS(
  annual_data,
  file.path(output_dir, "annual_demand_allocation.rds")
)

# Save the FoR dataset with its new reporting categories
saveRDS(
  for_data,
  file.path(output_dir, "field_of_research.rds")
)

# Verify that both output files were created
output_files <- c(
  file.path(output_dir, "annual_demand_allocation.rds"),
  file.path(output_dir, "field_of_research.rds")
)

stopifnot(all(file.exists(output_files)))

output_files
```

```
[1] "assets/data/data standardised/annual_demand_allocation.rds"
[2] "assets/data/data standardised/field_of_research.rds"
```

Chapter 7

Data Conversion

7.1 Purpose

The purpose of this step is to convert historical High-Performance Computing (HPC) demand and allocation values into more comparable computational resource measures across reporting years, facilities, and systems.

The integrated dataset covers 13 financial years, from 2013/14 to 2025/26, and contains observations reported in different accounting units, including kilo service units (kSU), million compute hours, CPU core-hours, and GPU-related service units.

These observations originate from different HPC facilities and systems, including the National Computational Infrastructure (NCI) and Pawsey Supercomputing Research Centre. Historical system information includes Raijin, Magnus, FlashLite, MASSIVE, Gadi, and Setonix CPU and GPU resources.

Service-unit accounting conventions can vary across facilities, systems, resource types, and reporting periods. Consequently, values expressed in the same unit, such as kSU, are not necessarily computationally equivalent.

This stage therefore evaluates conversion rules at the system and resource level rather than applying a single conversion factor across the historical dataset.

Where sufficient documentation is available, original demand and allocation values are converted into CPU core-hours or GPU hours using the applicable accounting rules. Where conversion cannot be supported reliably, original values and their limitations are retained without introducing unsupported assumptions.

The conversion process preserves the original reported values, units, source references, derivation methods, and conversion assumptions to maintain transparency and reproducibility for subsequent exploratory analysis and forecasting.

7.2 Conversion Approach

The conversion process uses the standardised annual demand and allocation dataset produced in Chapter 04, together with the system-level resource information collected during Data Integration.

Each historical observation is assessed according to its reporting period, facility, HPC system, resource type, original accounting unit, and available conversion documentation.

The conversion process follows four principles:

1. **Preserve original observations.** Original demand and allocation values, units, source references, and derivation assumptions are retained without modification.
2. **Identify resource-specific information.** CPU and GPU resources are distinguished where supported by the original source records. Aggregate observations without a documented resource split remain unallocated to a specific resource type.
3. **Apply documented conversion rules.** Conversion factors are selected according to the relevant system, resource type, and reporting period. No universal service-unit-to-compute-hour factor is assumed.
4. **Record conversion uncertainty.** Where direct conversion rules or resource-level observations are unavailable, transparent assumptions are applied to produce estimated values. Each assumption is documented with its rationale, supporting evidence, confidence level, and limitations. Reported, derived, and assumed values remain distinguishable throughout the analysis.

The general relationship between kilo service units (kSU) and service units (SU) is:

$$\text{SU} = \text{kSU} \times 1000$$

However, conversion from service units to physical computational hours depends on the accounting rules of the relevant HPC system.

For example, CPU core-hours and GPU hours require separate conversion factors and cannot be combined directly into a single physical compute-hour measure.

The conversion framework therefore maintains separate demand and allocation measures and records conversion status, supporting evidence, and limitations for each observation.

Where system-level breakdowns are available, they will be assessed separately before aggregation. Historical observations lacking sufficient detail will not be assigned unsupported CPU/GPU proportions.

```r
library(dplyr)

# Load the standardised annual dataset from Chapter 04
annual_standardised <- readRDS(
  "assets/data/data standardised/annual_demand_allocation.rds"
)

# Inspect available variables before conversion
names(annual_standardised)
```

```
 [1] "year"              "source_period"     "requested_compute"
 [4] "allocated_compute" "unit"              "requested_basis"
 [7] "allocated_basis"   "requested_source"  "allocated_source"
[10] "derivation_note"   "source_url"
```

```r
# Confirm the expected reporting years
annual_standardised |>
  select(year, source_period, unit) |>
  arrange(year)
```

```
# A tibble: 13 x 3
    year source_period unit
   <dbl> <chr>         <chr>
 1  2014 2013/14       kSU
 2  2015 2014/15       mixed: million hours requested / kSU allocated
 3  2016 2015/16       mixed: million hours requested / kSU allocated
 4  2017 2016/17       million core hours
 5  2018 2017/18       million hours
 6  2019 2018/19       million CPU-hours
 7  2020 2019/20       kSU
 8  2021 2020/21       kSU
 9  2022 2021/22       kSU
10  2023 2022/23       kSU
11  2024 2023/24       kSU
12  2025 2024/25       kSU
13  2026 2025/26       kSU
```

```r
stopifnot(
  nrow(annual_standardised) == 13,
  all(sort(annual_standardised$year) == 2014:2026)
)
```

```r
# Create the conversion framework
# Original annual values remain unchanged

# Create conversion framework and retain existing kSU values
conversion_data <- annual_standardised |>
```

```
  mutate(
    demand_ksu = if_else(
      unit == "kSU",
      as.numeric(requested_compute),
      NA_real_
    ),

    allocation_ksu = if_else(
      unit == "kSU" |
        grepl("kSU allocated", unit, fixed = TRUE),
      as.numeric(allocated_compute),
      NA_real_
    ),

    demand_cpu_core_hours = NA_real_,
    allocation_cpu_core_hours = NA_real_,
    demand_gpu_hours = NA_real_,
    allocation_gpu_hours = NA_real_,

    conversion_status = case_when(
      unit == "kSU" ~ "Original kSU retained",
      grepl("kSU allocated", unit, fixed = TRUE) ~
        "Allocation kSU retained; demand conversion required",
      TRUE ~ "Historical unit conversion required"
    ),

    conversion_note = NA_character_
  )

conversion_data |>
  select(
    year,
    demand_ksu,
    allocation_ksu,
    conversion_status
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 4
    year demand_ksu allocation_ksu conversion_status
   <dbl>      <dbl>          <dbl> <chr>
 1  2014     131356          83505 Original kSU retained
 2  2015         NA          88731 Allocation kSU retained; demand conversion r~
 3  2016         NA          97112 Allocation kSU retained; demand conversion r~
 4  2017         NA             NA Historical unit conversion required
```

```
 5  2018        NA              NA Historical unit conversion required
 6  2019        NA              NA Historical unit conversion required
 7  2020    607980          255148 Original kSU retained
 8  2021   1193811          397937 Original kSU retained
 9  2022   1162230          581115 Original kSU retained
10  2023   1333240          817956 Original kSU retained
11  2024   1425560.         812569 Original kSU retained
12  2025   1564643.         876200 Original kSU retained
13  2026   2242817          872288 Original kSU retained
```

```
# Check the framework
conversion_data |>
  select(
    year,
    source_period,
    demand_ksu,
    allocation_ksu,
    demand_cpu_core_hours,
    allocation_cpu_core_hours,
    demand_gpu_hours,
    allocation_gpu_hours,
    conversion_status
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 13 x 9
    year source_period demand_ksu allocation_ksu demand_cpu_core_hours
   <dbl> <chr>              <dbl>          <dbl>                  <dbl>
 1  2014 2013/14          131356          83505                     NA
 2  2015 2014/15              NA          88731                     NA
 3  2016 2015/16              NA          97112                     NA
 4  2017 2016/17              NA             NA                     NA
 5  2018 2017/18              NA             NA                     NA
 6  2019 2018/19              NA             NA                     NA
 7  2020 2019/20          607980          255148                     NA
 8  2021 2020/21         1193811          397937                     NA
 9  2022 2021/22         1162230          581115                     NA
10  2023 2022/23         1333240          817956                     NA
11  2024 2023/24         1425560.         812569                     NA
12  2025 2024/25         1564643.         876200                     NA
13  2026 2025/26         2242817          872288                     NA
   allocation_cpu_core_hours demand_gpu_hours allocation_gpu_hours
                       <dbl>            <dbl>                <dbl>
 1                        NA               NA                   NA
 2                        NA               NA                   NA
 3                        NA               NA                   NA
```

```
 4                       NA                NA                    NA
 5                       NA                NA                    NA
 6                       NA                NA                    NA
 7                       NA                NA                    NA
 8                       NA                NA                    NA
 9                       NA                NA                    NA
10                       NA                NA                    NA
11                       NA                NA                    NA
12                       NA                NA                    NA
13                       NA                NA                    NA
   conversion_status
   <chr>
 1 Original kSU retained
 2 Allocation kSU retained; demand conversion required
 3 Allocation kSU retained; demand conversion required
 4 Historical unit conversion required
 5 Historical unit conversion required
 6 Historical unit conversion required
 7 Original kSU retained
 8 Original kSU retained
 9 Original kSU retained
10 Original kSU retained
11 Original kSU retained
12 Original kSU retained
13 Original kSU retained
```

```r
# Confirm no historical observations were lost
stopifnot(
  nrow(conversion_data) == nrow(annual_standardised),
  identical(
    conversion_data$year,
    annual_standardised$year
  )
)
```

```r
# Review original historical values and supporting evidence
historical_conversion_check <- conversion_data |>
  filter(year >= 2014, year <= 2019) |>
  select(
    year,
    source_period,
    requested_compute,
    allocated_compute,
    unit,
    requested_basis,
    allocated_basis,
```

```
    derivation_note
  )

historical_conversion_check |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 6 x 8
   year source_period requested_compute allocated_compute
  <dbl> <chr>                    <dbl>             <dbl>
1  2014 2013/14                131356              83505
2  2015 2014/15                   202.             88731
3  2016 2015/16                   225              97112
4  2017 2016/17                   200                100
5  2018 2017/18                   529.               222
6  2019 2018/19                   660                220
  unit                                          requested_basis allocated_basis
  <chr>                                         <chr>           <chr>
1 kSU                                           Direct          Direct
2 mixed: million hours requested / kSU allocated Derived        Direct
3 mixed: million hours requested / kSU allocated Derived        Direct
4 million core hours                            Proxy           Capacity proxy
5 million hours                                 Derived         Direct
6 million CPU-hours                             Derived proxy   Capacity proxy
  derivation_note
  <chr>
1 <NA>
2 Requested compute derived from reported allocation and demand relationship.
3 Requested compute derived from reported one-third fulfilment relationship.
4 Requested compute retained as a lower-bound proxy; available capacity used as~
5 Requested compute derived from 222 million hours and a 42% fulfilment relatio~
6 Requested compute derived as 220 × 3; available capacity used as the allocati~
```

```
# Historical conversion assumptions
# Baseline: 1 service unit per CPU core-hour
# 1 kSU = 1,000 SU
# 1 million hours = 1,000,000 hours

historical_su_per_cpu_hour <- 1

conversion_data <- conversion_data |>
  mutate(
    demand_ksu = case_when(
      year %in% 2015:2019 ~
        requested_compute * 1000 * historical_su_per_cpu_hour,
      TRUE ~ demand_ksu
    ),
```

```
    allocation_ksu = case_when(
      year %in% 2017:2019 ~
        allocated_compute * 1000 * historical_su_per_cpu_hour,
      TRUE ~ allocation_ksu
    ),

    conversion_status = case_when(
      year == 2014 ~ "Reported kSU; resource conversion assumed",
      year %in% 2015:2016 ~
        "Demand kSU estimated; allocation kSU reported",
      year %in% c(2017, 2019) ~
        "Both kSU estimated; capacity proxy used",
      year == 2018 ~
        "Both kSU estimated; allocation reported",
      TRUE ~ conversion_status
    ),

    conversion_note = case_when(
      year == 2014 ~
        "Original reported kSU retained; CPU/GPU resource split not established.",
      year %in% 2015:2016 ~
        "Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU retain
      year == 2017 ~
        "Million core-hours converted using assumed 1 SU/core-hour; capacity is an allocatio
      year == 2018 ~
        "Million hours converted using assumed 1 SU/hour; original rounded values retained."
      year == 2019 ~
        "Million CPU-hours converted using assumed 1 SU/CPU-hour; available capacity is an a
      TRUE ~ conversion_note
    )
  )

# Display converted historical totals
conversion_data |>
  filter(year <= 2019) |>
  select(
    source_period,
    demand_ksu,
    allocation_ksu,
    conversion_status,
    conversion_note
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 6 x 5
```

```
  source_period demand_ksu allocation_ksu
  <chr>               <dbl>          <dbl>
1 2013/14          131356           83505
2 2014/15          202500           88731
3 2015/16          225000           97112
4 2016/17          200000          100000
5 2017/18          528571.         222000
6 2018/19          660000          220000
  conversion_status
  <chr>
1 Reported kSU; resource conversion assumed
2 Demand kSU estimated; allocation kSU reported
3 Demand kSU estimated; allocation kSU reported
4 Both kSU estimated; capacity proxy used
5 Both kSU estimated; allocation reported
6 Both kSU estimated; capacity proxy used
  conversion_note
  <chr>
1 Original reported kSU retained; CPU/GPU resource split not established.
2 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU ~
3 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU ~
4 Million core-hours converted using assumed 1 SU/core-hour; capacity is an all~
5 Million hours converted using assumed 1 SU/hour; original rounded values reta~
6 Million CPU-hours converted using assumed 1 SU/CPU-hour; available capacity i~
```

```r
conversion_data |>
  filter(year <= 2019) |>
  select(
    source_period,
    demand_ksu,
    allocation_ksu,
    requested_basis,
    allocated_basis,
    source_url,
    conversion_note
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 6 x 7
  source_period demand_ksu allocation_ksu requested_basis allocated_basis
  <chr>               <dbl>          <dbl> <chr>           <chr>
1 2013/14          131356           83505 Direct          Direct
2 2014/15          202500           88731 Derived         Direct
3 2015/16          225000           97112 Derived         Direct
4 2016/17          200000          100000 Proxy           Capacity proxy
5 2017/18          528571.         222000 Derived         Direct
```

```
6 2018/19          660000          220000 Derived proxy   Capacity proxy
  source_url
  <chr>
1 https://nci.org.au/sites/default/files/annual-reports/2019-
07/About%20Us_Annu~
2 https://nci.org.au/sites/default/files/annual-reports/2019-
07/About%20Us_Annu~
3 https://nci.org.au/sites/default/files/annual-reports/2019-
07/About%20Us_Annu~
4 https://pawsey.org.au/wp-content/uploads/2018/10/Pawsey-Annual-
Report-2016-20~
5 https://nci.org.au/news-events/news/ncmas-allocations-now-
available-online
6 https://nci.org.au/sites/default/files/documents/2019-10/NCMAS-
2020-Informati~
  conversion_note
  <chr>
1 Original reported kSU retained; CPU/GPU resource split not established.
2 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU ~
3 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU ~
4 Million core-hours converted using assumed 1 SU/core-hour; capacity is an all~
5 Million hours converted using assumed 1 SU/hour; original rounded values reta~
6 Million CPU-hours converted using assumed 1 SU/CPU-hour; available capacity i~
```

7.3 CPU Resources

CPU resource conversion translates historical service-unit allocations and reported compute hours into CPU core-hours.

The conversion is performed at the HPC system level where sufficient information is available. This is necessary because the accounting relationship between service units and CPU core-hours may differ across facilities and reporting periods.

For observations reported in kSU, CPU core-hours are calculated as:

$$\text{CPU core-hours} = \frac{\text{CPU kSU} \times 1000}{\text{SU per CPU core-hour}}$$

For observations already reported in CPU core-hours, the original values are retained without applying an additional service-unit conversion.

Where aggregate kSU values include both CPU and GPU resources, the CPU component must first be identified or estimated before the conversion formula is applied.

The conversion framework considers the historical systems identified during Data Integration, including Raijin, Magnus, FlashLite, MASSIVE, Gadi, and Setonix

CPU.

Documented accounting factors are prioritised. Where system-specific factors or resource splits are unavailable, assumptions are applied with explicit justification, source references, and confidence classifications.

All calculations preserve the original observations and distinguish directly reported CPU values from derived and assumed estimates.

```
# Historical CPU core-hour conversion: 2013/14-2018/19
# Assumption: 1 SU per CPU core-hour for historical CPU kSU
# Years already reported in millions of hours are scaled by 1,000,000

historical_cpu_share <- 1.0  # Baseline assumption: 100% CPU-equivalent

conversion_data <- conversion_data |>
  mutate(
    demand_cpu_core_hours = case_when(
      year == 2014 ~ demand_ksu * 1000 * historical_cpu_share,
      year %in% 2015:2019 ~ requested_compute * 1e6,
      TRUE ~ demand_cpu_core_hours
    ),

    allocation_cpu_core_hours = case_when(
      year %in% c(2014, 2015, 2016) ~
        allocation_ksu * 1000 * historical_cpu_share,
      year %in% 2017:2019 ~ allocated_compute * 1e6,
      TRUE ~ allocation_cpu_core_hours
    ),

    cpu_conversion_assumption = case_when(
      year == 2014 ~
        "Assumed 100% CPU-equivalent; 1 SU per CPU core-hour. Historical Fornax GPU may be i
      year %in% 2015:2016 ~
        "Assumed reported million hours represent CPU core-hours; aggregate kSU allocation t
      year == 2017 ~
        "Reported million core-hours converted directly; allocation represents available cap
      year == 2018 ~
        "Assumed reported million hours represent CPU core-hours; original system mix not fu
      year == 2019 ~
        "Reported million CPU-hours converted directly; allocation represents available capa
      TRUE ~ NA_character_
    )
  )

conversion_data |>
  filter(year <= 2019) |>
```

```
  select(
    source_period,
    demand_cpu_core_hours,
    allocation_cpu_core_hours,
    cpu_conversion_assumption
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 6 x 4
  source_period demand_cpu_core_hours allocation_cpu_core_hours
  <chr>                         <dbl>                     <dbl>
1 2013/14                   131356000                  83505000
2 2014/15                   202500000                  88731000
3 2015/16                   225000000                  97112000
4 2016/17                   200000000                 100000000
5 2017/18                   528571429.                222000000
6 2018/19                   660000000                 220000000
  cpu_conversion_assumption
  <chr>
1 Assumed 100% CPU-equivalent; 1 SU per CPU core-hour. Historical Fornax GPU ma~
2 Assumed reported million hours represent CPU core-hours; aggregate kSU alloca~
3 Assumed reported million hours represent CPU core-hours; aggregate kSU alloca~
4 Reported million core-hours converted directly; allocation represents availab~
5 Assumed reported million hours represent CPU core-hours; original system mix ~
6 Reported million CPU-hours converted directly; allocation represents availabl~
```

```
# System-level CPU allocation data
# Analytical year 2021 = FY2020/21
# Values are in kSU

cpu_allocation_systems <- tibble::tribble(
  ~year, ~system,       ~cpu_ksu, ~su_per_core_hour,
  2021,  "Magnus",         107730, 1,
  2021,  "FlashLite",         992, 1,
  2021,  "MASSIVE",          2135, 1,
  2021,  "Gadi",           287080, 2,
  2022,  "MASSIVE",          1845, 1,
  2022,  "Gadi",           300120, 2,
  2022,  "Setonix CPU",    108700, 1,
  2023,  "Gadi",           328156, 2,
  2023,  "Setonix CPU",    327400, 1,
  2024,  "Gadi",           305784, 2,
  2024,  "Setonix CPU",    339155, 1,
  2025,  "Gadi",           355200, 2,
  2025,  "Setonix CPU",    349950, 1,
  2026,  "Gadi",           387288, 2,
```

```
  2026,  "Setonix CPU",   325000, 1
)

# Calculate CPU core-hours for each system
cpu_allocation_systems <- cpu_allocation_systems |>
  mutate(
    cpu_core_hours =
      cpu_ksu * 1000 / su_per_core_hour,

    factor_status = case_when(
      system %in% c("FlashLite", "MASSIVE") ~
        "Provisional 1 SU/core-hour assumption",
      TRUE ~
        "Working system-specific factor from conversion workbook"
    )
  )

# Aggregate CPU core-hours by year
cpu_allocation_annual <- cpu_allocation_systems |>
  group_by(year) |>
  summarise(
    allocation_cpu_core_hours =
      sum(cpu_core_hours),
    allocation_cpu_ksu =
      sum(cpu_ksu),
    .groups = "drop"
  )

cpu_allocation_annual |>
  print(n = Inf)
```

```
# A tibble: 6 x 3
   year allocation_cpu_core_hours allocation_cpu_ksu
  <dbl>                     <dbl>              <dbl>
1  2021                 254397000             397937
2  2022                 260605000             410665
3  2023                 491478000             655556
4  2024                 492047000             644939
5  2025                 527550000             705150
6  2026                 518644000             712288
```

```
# Compare system-level CPU allocation with annual totals

cpu_allocation_check <- cpu_allocation_annual |>
  left_join(
    conversion_data |>
```

```
      select(year, source_period, allocation_ksu),
    by = "year"
  ) |>
  mutate(
    non_cpu_ksu =
      allocation_ksu - allocation_cpu_ksu
  )

cpu_allocation_check |>
  select(
    source_period,
    allocation_ksu,
    allocation_cpu_ksu,
    non_cpu_ksu
  ) |>
  print(n = Inf)
```

```
# A tibble: 6 x 4
  source_period allocation_ksu allocation_cpu_ksu non_cpu_ksu
  <chr>                  <dbl>              <dbl>       <dbl>
1 2020/21               397937             397937           0
2 2021/22               581115             410665      170450
3 2022/23               817956             655556      162400
4 2023/24               812569             644939      167630
5 2024/25               876200             705150      171050
6 2025/26               872288             712288      160000
```

```
stopifnot(
  all(cpu_allocation_check$non_cpu_ksu >= 0)
)
```

```
# Integrate system-level CPU allocations
# Preserve existing historical CPU values

conversion_data <- conversion_data |>
  left_join(
    cpu_allocation_annual |>
      select(
        year,
        allocation_cpu_core_hours
      ) |>
      rename(
        calculated_allocation_cpu_core_hours =
          allocation_cpu_core_hours
      ),
    by = "year"
```

```
  ) |>
  mutate(
    allocation_cpu_core_hours = coalesce(
      calculated_allocation_cpu_core_hours,
      allocation_cpu_core_hours
    )
  ) |>
  select(-calculated_allocation_cpu_core_hours)

# Complete FY2019/20 CPU allocation
# Assumption: 100% Gadi CPU-equivalent
# Working factor: 2 SU per CPU core-hour

conversion_data <- conversion_data |>
  mutate(
    allocation_cpu_core_hours = case_when(
      year == 2020 ~ allocation_ksu * 1000 / 2,
      TRUE ~ allocation_cpu_core_hours
    ),

    cpu_conversion_assumption = case_when(
      year == 2020 ~ paste(
        "FY2019/20: aggregate allocation assumed to be",
        "100% Gadi CPU-equivalent at 2 SU/core-hour.",
        "System split unavailable; sensitivity analysis required."
      ),
      TRUE ~ cpu_conversion_assumption
    )
  )

# Display results AFTER correction
conversion_data |>
  select(
    source_period,
    allocation_cpu_core_hours
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 2
   source_period allocation_cpu_core_hours
   <chr>                             <dbl>
 1 2013/14                        83505000
 2 2014/15                        88731000
 3 2015/16                        97112000
 4 2016/17                       100000000
```

```
 5 2017/18                       222000000
 6 2018/19                       220000000
 7 2019/20                       127574000
 8 2020/21                       254397000
 9 2021/22                       260605000
10 2022/23                       491478000
11 2023/24                       492047000
12 2024/25                       527550000
13 2025/26                       518644000
```

```r
# Validate all 13 years
stopifnot(
  nrow(conversion_data) == 13,
  all(!is.na(conversion_data$allocation_cpu_core_hours))
)
```

```r
# CPU demand conversion: FY2019/20-FY2025/26
# Preserve original annual demand and allocation values

# 1. Direct system-level CPU demand (kSU)
cpu_demand_systems <- tibble::tribble(
  ~year, ~system,       ~cpu_ksu, ~su_per_core_hour,
  2023,  "Gadi",          770866, 2,
  2023,  "Setonix CPU",   390033, 1,
  2026,  "Gadi",         1246722, 2,
  2026,  "Setonix CPU",   611675, 1
)

cpu_demand_direct <- cpu_demand_systems |>
  mutate(
    cpu_core_hours = cpu_ksu * 1000 / su_per_core_hour
  ) |>
  group_by(year) |>
  summarise(
    demand_cpu_ksu = sum(cpu_ksu),
    demand_cpu_core_hours = sum(cpu_core_hours),
    .groups = "drop"
  )

# 2. Estimate CPU demand using allocation composition
# FY2019/20: assume 100% Gadi CPU-equivalent at 2 SU/core-hour
# Other years: assume demand follows allocation resource/system mix

cpu_demand_estimated <- conversion_data |>
  filter(year >= 2020) |>
  left_join(
```

```
    cpu_allocation_annual |>
      select(year, allocation_cpu_ksu),
    by = "year"
  ) |>
  mutate(
    allocation_cpu_ksu = case_when(
      year == 2020 ~ allocation_ksu,
      TRUE ~ allocation_cpu_ksu
    ),

    cpu_allocation_share =
      allocation_cpu_ksu / allocation_ksu,

    estimated_demand_cpu_ksu =
      demand_ksu * cpu_allocation_share,

    estimated_demand_cpu_core_hours = case_when(
      year == 2020 ~
        estimated_demand_cpu_ksu * 1000 / 2,

      TRUE ~
        estimated_demand_cpu_ksu *
        (allocation_cpu_core_hours / allocation_cpu_ksu)
    )
  ) |>
  select(
    year,
    estimated_demand_cpu_ksu,
    estimated_demand_cpu_core_hours,
    cpu_allocation_share
  )

# 3. Prioritise direct system-level demand over estimates

cpu_demand_annual <- cpu_demand_estimated |>
  left_join(
    cpu_demand_direct,
    by = "year"
  ) |>
  mutate(
    demand_cpu_ksu = coalesce(
      demand_cpu_ksu,
      estimated_demand_cpu_ksu
    ),
```

```
    demand_cpu_core_hours = coalesce(
      demand_cpu_core_hours,
      estimated_demand_cpu_core_hours
    ),

    cpu_demand_method = case_when(
      year %in% c(2023, 2026) ~
        "Reported CPU system split; working SU conversion factors",

      year == 2020 ~
        "Assumed 100% Gadi CPU-equivalent; 2 SU/core-hour",

      TRUE ~
        "Estimated using allocation CPU share and weighted CPU conversion"
    ),

    cpu_demand_confidence = case_when(
      year %in% c(2023, 2026) ~ "Higher: reported resource split",
      year == 2020 ~ "Low: assumed CPU-only system mix",
      TRUE ~ "Moderate-low: allocation mix proxy"
    ),

    cpu_demand_assumption = case_when(
      year %in% c(2023, 2026) ~
        "CPU demand split is reported; physical hours use working system factors.",

      year == 2020 ~
        "Aggregate demand assumed entirely Gadi CPU at 2 SU/core-hour; GPU demand not indepe

      TRUE ~
        "CPU demand share and system mix assumed equal to allocation composition; actual rec
    )
  )

# 4. Validate all seven years

stopifnot(
  nrow(cpu_demand_annual) == 7,
  all(!is.na(cpu_demand_annual$demand_cpu_ksu)),
  all(!is.na(cpu_demand_annual$demand_cpu_core_hours))
)

# 5. Display results

cpu_demand_annual |>
```

```
  select(
    year,
    demand_cpu_ksu,
    demand_cpu_core_hours,
    cpu_demand_method,
    cpu_demand_confidence
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 7 x 5
   year demand_cpu_ksu demand_cpu_core_hours
  <dbl>          <dbl>                <dbl>
1  2020         607980            303990000
2  2021        1193811            763191000
3  2022         821330            521210000
4  2023        1160899            775466000
5  2024        1131472.           863240351.
6  2025        1259196.           942053571.
7  2026        1858397           1235036000
  cpu_demand_method
  <chr>
1 Assumed 100% Gadi CPU-equivalent; 2 SU/core-hour
2 Estimated using allocation CPU share and weighted CPU conversion
3 Estimated using allocation CPU share and weighted CPU conversion
4 Reported CPU system split; working SU conversion factors
5 Estimated using allocation CPU share and weighted CPU conversion
6 Estimated using allocation CPU share and weighted CPU conversion
7 Reported CPU system split; working SU conversion factors
  cpu_demand_confidence
  <chr>
1 Low: assumed CPU-only system mix
2 Moderate-low: allocation mix proxy
3 Moderate-low: allocation mix proxy
4 Higher: reported resource split
5 Moderate-low: allocation mix proxy
6 Moderate-low: allocation mix proxy
7 Higher: reported resource split
```

```
# Integrate CPU demand into the main conversion dataset
# Preserve historical CPU values and original source observations

conversion_data <- conversion_data |>
  left_join(
    cpu_demand_annual |>
      select(
        year,
```

```
        demand_cpu_core_hours,
        cpu_demand_method,
        cpu_demand_confidence,
        cpu_demand_assumption
      ) |>
      rename(
        calculated_demand_cpu_core_hours =
          demand_cpu_core_hours
      ),
    by = "year"
  ) |>
  mutate(
    demand_cpu_core_hours = coalesce(
      calculated_demand_cpu_core_hours,
      demand_cpu_core_hours
    )
  ) |>
  select(-calculated_demand_cpu_core_hours)

# Validate CPU conversion for all 13 years

stopifnot(
  nrow(conversion_data) == 13,
  all(!is.na(conversion_data$demand_cpu_core_hours)),
  all(!is.na(conversion_data$allocation_cpu_core_hours))
)

# Display consolidated CPU results

conversion_data |>
  select(
    year,
    source_period,
    demand_cpu_core_hours,
    allocation_cpu_core_hours
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 4
    year source_period demand_cpu_core_hours allocation_cpu_core_hours
   <dbl> <chr>                         <dbl>                     <dbl>
 1  2014 2013/14                  131356000                  83505000
 2  2015 2014/15                  202500000                  88731000
 3  2016 2015/16                  225000000                  97112000
 4  2017 2016/17                  200000000                 100000000
```

5	2018	2017/18	528571429.	222000000
6	2019	2018/19	660000000	220000000
7	2020	2019/20	303990000	127574000
8	2021	2020/21	763191000	254397000
9	2022	2021/22	521210000	260605000
10	2023	2022/23	775466000	491478000
11	2024	2023/24	863240351.	492047000
12	2025	2024/25	942053571.	527550000
13	2026	2025/26	1235036000	518644000

7.4 GPU Resources

GPU resource conversion translates GPU-specific service-unit allocations and demand into GPU-hours.

Unlike CPU core-hours, GPU-hours represent the use of GPU resources and depend on the accounting conventions of the relevant HPC system.

For GPU observations reported in kSU, the conversion is:

$$\text{GPU hours} = \frac{\text{GPU kSU} \times 1000}{\text{SU per GPU hour}}$$

The analysis prioritises directly reported GPU demand and allocation figures. Where GPU demand is unavailable, estimates are developed using documented assumptions and available resource-level evidence.

The conversion distinguishes reported GPU observations from estimated values. Historical years without a documented GPU breakdown require separate assessment rather than being automatically classified as having zero GPU demand.

Original source values, conversion factors, assumptions and limitations are preserved for reproducibility.

```
# GPU allocation: FY2021/22-FY2025/26
# Source: reconciled system-level allocation evidence
# Working conversion factor: 128 SU per GPU-hour

gpu_allocation_systems <- tibble::tribble(
  ~year, ~system,       ~gpu_ksu, ~su_per_gpu_hour,
  2022,  "Setonix GPU",    170450, 128,
  2023,  "Setonix GPU",    162400, 128,
  2024,  "Setonix GPU",    167630, 128,
  2025,  "Setonix GPU",    171050, 128,
  2026,  "Setonix GPU",    160000, 128
)
```

```
gpu_allocation_annual <- gpu_allocation_systems |>
  mutate(
    allocation_gpu_hours =
      gpu_ksu * 1000 / su_per_gpu_hour
  ) |>
  group_by(year) |>
  summarise(
    allocation_gpu_ksu = sum(gpu_ksu),
    allocation_gpu_hours = sum(allocation_gpu_hours),
    .groups = "drop"
  )

# Check GPU allocation against annual non-CPU residuals
gpu_allocation_check <- cpu_allocation_check |>
  left_join(
    gpu_allocation_annual,
    by = "year"
  ) |>
  filter(year >= 2022) |>
  mutate(
    reconciliation_difference =
      non_cpu_ksu - allocation_gpu_ksu
  )

stopifnot(
  all(gpu_allocation_check$reconciliation_difference == 0)
)

# Display GPU allocation results
gpu_allocation_annual |>
  print(n = Inf)
```

```
# A tibble: 5 x 3
   year allocation_gpu_ksu allocation_gpu_hours
  <dbl>              <dbl>                <dbl>
1  2022             170450             1331641.
2  2023             162400             1268750
3  2024             167630             1309609.
4  2025             171050             1336328.
5  2026             160000             1250000
```

```
# GPU demand: FY2021/22-FY2025/26
# Working factor: 128 SU per Setonix GPU-hour

gpu_demand_direct <- tibble::tribble(
  ~year, ~demand_gpu_ksu,
```

```
  2023, 172341,
  2026, 384420
)

gpu_demand_annual <- conversion_data |>
  filter(year %in% 2022:2026) |>
  left_join(
    gpu_allocation_annual |>
      select(year, allocation_gpu_ksu),
    by = "year"
  ) |>
  left_join(
    gpu_demand_direct,
    by = "year"
  ) |>
  mutate(
    gpu_allocation_share =
      allocation_gpu_ksu / allocation_ksu,

    estimated_demand_gpu_ksu =
      demand_ksu * gpu_allocation_share,

    demand_gpu_ksu = coalesce(
      demand_gpu_ksu,
      estimated_demand_gpu_ksu
    ),

    demand_gpu_hours =
      demand_gpu_ksu * 1000 / 128,

    gpu_demand_method = if_else(
      year %in% c(2023, 2026),
      "Reported Setonix GPU demand; working conversion factor",
      "Estimated using GPU allocation share"
    ),

    gpu_demand_confidence = if_else(
      year %in% c(2023, 2026),
      "Higher: reported GPU demand",
      "Moderate-low: allocation share assumption"
    )
  ) |>
  select(
    year,
    demand_gpu_ksu,
```

```
    demand_gpu_hours,
    gpu_demand_method,
    gpu_demand_confidence
  )

# Validate GPU demand
stopifnot(
  nrow(gpu_demand_annual) == 5,
  all(!is.na(gpu_demand_annual$demand_gpu_hours))
)

gpu_demand_annual |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 5 x 5
   year demand_gpu_ksu demand_gpu_hours
  <dbl>          <dbl>            <dbl>
1  2022         340900          2663281.
2  2023         172341          1346414.
3  2024         294088.         2297560.
4  2025         305446.         2386300.
5  2026         384420          3003281.
  gpu_demand_method
  <chr>
1 Estimated using GPU allocation share
2 Reported Setonix GPU demand; working conversion factor
3 Estimated using GPU allocation share
4 Estimated using GPU allocation share
5 Reported Setonix GPU demand; working conversion factor
  gpu_demand_confidence
  <chr>
1 Moderate-low: allocation share assumption
2 Higher: reported GPU demand
3 Moderate-low: allocation share assumption
4 Moderate-low: allocation share assumption
5 Higher: reported GPU demand
```

```
# Reconcile CPU and GPU demand against annual demand

resource_demand_check <- cpu_demand_annual |>
  select(year, demand_cpu_ksu) |>
  inner_join(
    gpu_demand_annual |>
      select(year, demand_gpu_ksu),
    by = "year"
  ) |>
```

```
  left_join(
    conversion_data |>
      select(year, demand_ksu),
    by = "year"
  ) |>
  mutate(
    combined_demand_ksu =
      demand_cpu_ksu + demand_gpu_ksu,

    reconciliation_difference =
      demand_ksu - combined_demand_ksu
  )

resource_demand_check |>
  print(n = Inf)
```

```
# A tibble: 5 x 6
   year demand_cpu_ksu demand_gpu_ksu demand_ksu combined_demand_ksu
  <dbl>          <dbl>          <dbl>      <dbl>               <dbl>
1  2022         821330         340900    1162230             1162230
2  2023        1160899         172341    1333240             1333240
3  2024        1131472.        294088.   1425560.            1425560.
4  2025        1259196.        305446.   1564643.            1564643.
5  2026        1858397         384420    2242817             2242817
# i 1 more variable: reconciliation_difference <dbl>
```

```
stopifnot(
  all(abs(resource_demand_check$reconciliation_difference) < 1e-6)
)
```

```
# Integrate GPU demand and allocation into the annual conversion dataset
# Preserve original values and existing CPU calculations

gpu_conversion_annual <- gpu_allocation_annual |>
  full_join(
    gpu_demand_annual |>
      select(
        year,
        demand_gpu_ksu,
        demand_gpu_hours,
        gpu_demand_method,
        gpu_demand_confidence
      ),
    by = "year"
  )
```

```
conversion_data <- conversion_data |>
  left_join(
    gpu_conversion_annual |>
      rename(
        calculated_demand_gpu_hours = demand_gpu_hours,
        calculated_allocation_gpu_hours = allocation_gpu_hours
      ),
    by = "year"
  ) |>
  mutate(
    demand_gpu_hours = coalesce(
      calculated_demand_gpu_hours,
      demand_gpu_hours
    ),
    allocation_gpu_hours = coalesce(
      calculated_allocation_gpu_hours,
      allocation_gpu_hours
    )
  ) |>
  select(
    -calculated_demand_gpu_hours,
    -calculated_allocation_gpu_hours
  )

# Check converted GPU values
conversion_data |>
  select(
    year,
    source_period,
    demand_gpu_hours,
    allocation_gpu_hours
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 4
    year source_period demand_gpu_hours allocation_gpu_hours
   <dbl> <chr>                    <dbl>                <dbl>
 1  2014 2013/14                     NA                   NA
 2  2015 2014/15                     NA                   NA
 3  2016 2015/16                     NA                   NA
 4  2017 2016/17                     NA                   NA
 5  2018 2017/18                     NA                   NA
 6  2019 2018/19                     NA                   NA
 7  2020 2019/20                     NA                   NA
 8  2021 2020/21                     NA                   NA
```

```
 9  2022 2021/22              2663281.            1331641.
10  2023 2022/23              1346414.            1268750
11  2024 2023/24              2297560.            1309609.
12  2025 2024/25              2386300.            1336328.
13  2026 2025/26              3003281.            1250000
```

```
# Validate years with reported Setonix GPU allocations
stopifnot(
  nrow(conversion_data) == 13,
  all(!is.na(
    conversion_data$demand_gpu_hours[
      conversion_data$year %in% 2022:2026
    ]
  )),
  all(!is.na(
    conversion_data$allocation_gpu_hours[
      conversion_data$year %in% 2022:2026
    ]
  ))
)
```

```
# Historical GPU estimation framework: FY2013/14-FY2020/21
# Assumptions must be documented and supported by historical evidence

historical_gpu_assumptions <- tibble::tibble(
  year = 2014:2021,

  gpu_share_assumed = c(
    NA_real_,  # 2013/14 - Fornax evidence requires assessment
    NA_real_,  # 2014/15
    NA_real_,  # 2015/16
    NA_real_,  # 2016/17
    NA_real_,  # 2017/18
    NA_real_,  # 2018/19
    NA_real_,  # 2019/20
    NA_real_   # 2020/21 - FlashLite evidence requires assessment
  ),

  su_per_gpu_hour = NA_real_,

  assumption_status = "Requires source-informed estimate"
)

historical_gpu_assumptions
```

```
# A tibble: 8 x 4
   year gpu_share_assumed su_per_gpu_hour assumption_status
```

```
  <int>             <dbl>           <dbl> <chr>
1  2014                NA              NA Requires source-informed estimate
2  2015                NA              NA Requires source-informed estimate
3  2016                NA              NA Requires source-informed estimate
4  2017                NA              NA Requires source-informed estimate
5  2018                NA              NA Requires source-informed estimate
6  2019                NA              NA Requires source-informed estimate
7  2020                NA              NA Requires source-informed estimate
8  2021                NA              NA Requires source-informed estimate
```

```
historical_gpu_evidence <- tibble::tribble(
  ~year, ~system, ~evidence_type, ~reported_ksu,
  ~source_url,

  2014, "Fornax",
  "2014 NCMAS GPU capacity available",
  1440,
  "https://nci.org.au/news-events/news/ncmas-information-applicants",

  2021, "FlashLite",
  "System included in 2021 NCMAS; GPU split not reported",
  NA_real_,
  "https://pawsey.org.au/ncmas2021/"
)

historical_gpu_evidence |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 2 x 5
   year system    evidence_type
  <dbl> <chr>     <chr>
1  2014 Fornax    2014 NCMAS GPU capacity available
2  2021 FlashLite System included in 2021 NCMAS; GPU split not reported
  reported_ksu source_url
         <dbl> <chr>
1         1440 https://nci.org.au/news-events/news/ncmas-information-
applicants
2           NA https://pawsey.org.au/ncmas2021/
```

```
# Historical system classification based on UQ documentation

flashlite_classification <- tibble::tibble(
  year = 2021,
  system = "FlashLite",
  allocation_ksu = 992,
  resource_class = "CPU",
  gpu_count = 0L,
```

```
  gpu_allocation_ksu = 0,
  evidence_status = "Verified system specification",
  source_url = paste0(
    "https://rcc.uq.edu.au/files/2009/",
    "FlashLite_User_Guide_20200122.pdf"
  )
)

flashlite_classification
```

```
# A tibble: 1 x 8
   year system    allocation_ksu resource_class gpu_count gpu_allocation_ksu
  <dbl> <chr>              <dbl> <chr>              <int>               <dbl>
1  2021 FlashLite            992 CPU                    0                   0
# i 2 more variables: evidence_status <chr>, source_url <chr>
```

```
# Gadi historical GPU conversion reference
# Charging factor from Pawsey's Gadi/Setonix comparison

gadi_gpu_reference <- tibble::tibble(
  year = 2020:2021,
  system = "Gadi GPU (NVIDIA V100)",
  su_per_gpu_hour = 36,
  conversion_formula = "GPU kSU * 1000 / 36",
  allocation_gpu_ksu = NA_real_,
  allocation_gpu_hours = NA_real_,
  evidence_status = paste(
    "GPU charging factor documented;",
    "historical GPU allocation split unavailable"
  ),
  source_url = paste0(
    "https://pawsey.atlassian.net/wiki/spaces/US/pages/",
    "51927586/Changes+in+Supercomputing+Services+for+2023"
  )
)

gadi_gpu_reference |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 2 x 8
   year system                 su_per_gpu_hour conversion_formula
  <int> <chr>                            <dbl> <chr>
1  2020 Gadi GPU (NVIDIA V100)              36 GPU kSU * 1000 / 36
2  2021 Gadi GPU (NVIDIA V100)              36 GPU kSU * 1000 / 36
  allocation_gpu_ksu allocation_gpu_hours
               <dbl>                <dbl>
```

```
1                 NA                    NA
2                 NA                    NA
  evidence_status
  <chr>
1 GPU charging factor documented; historical GPU allocation split unavailable
2 GPU charging factor documented; historical GPU allocation split unavailable
  source_url
  <chr>
1 https://pawsey.atlassian.net/wiki/spaces/US/pages/51927586/Changes+in+Superco~
2 https://pawsey.atlassian.net/wiki/spaces/US/pages/51927586/Changes+in+Superco~
```

```r
# Historical GPU conversion status
# Missing GPU-hours are unknown, not zero.

historical_gpu_status <- tibble::tibble(
  year = 2014:2021,
  gpu_system_evidence = c(
    "Fornax",
    "Historical GPU split not established",
    "Historical GPU split not established",
    "Historical GPU split not established",
    "Historical GPU split not established",
    "Historical GPU split not established",
    "Gadi",
    "Gadi; FlashLite classified as CPU"
  ),
  gpu_allocation_status = c(
    "Fornax capacity reported; allocation unconfirmed",
    rep("GPU allocation split unavailable", 5),
    "Gadi GPU allocation split unavailable",
    "Gadi GPU allocation split unavailable"
  ),
  gpu_conversion_status = "Not estimable from available allocation data",
  gpu_estimation_method = "No unsupported GPU share assigned"
)

# Check historical coverage
stopifnot(
  nrow(historical_gpu_status) == 8,
  identical(historical_gpu_status$year, 2014:2021)
)

historical_gpu_status |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 8 x 5
```

```
   year gpu_system_evidence
  <int> <chr>
1  2014 Fornax
2  2015 Historical GPU split not established
3  2016 Historical GPU split not established
4  2017 Historical GPU split not established
5  2018 Historical GPU split not established
6  2019 Historical GPU split not established
7  2020 Gadi
8  2021 Gadi; FlashLite classified as CPU
  gpu_allocation_status
  <chr>
1 Fornax capacity reported; allocation unconfirmed
2 GPU allocation split unavailable
3 GPU allocation split unavailable
4 GPU allocation split unavailable
5 GPU allocation split unavailable
6 GPU allocation split unavailable
7 Gadi GPU allocation split unavailable
8 Gadi GPU allocation split unavailable
  gpu_conversion_status                          gpu_estimation_method
  <chr>                                          <chr>
1 Not estimable from available allocation data No unsupported GPU share assigned
2 Not estimable from available allocation data No unsupported GPU share assigned
3 Not estimable from available allocation data No unsupported GPU share assigned
4 Not estimable from available allocation data No unsupported GPU share assigned
5 Not estimable from available allocation data No unsupported GPU share assigned
6 Not estimable from available allocation data No unsupported GPU share assigned
7 Not estimable from available allocation data No unsupported GPU share assigned
8 Not estimable from available allocation data No unsupported GPU share assigned
```

7.4.1 Historical GPU Estimation Scenarios

Historical GPU allocation and demand are estimated using low, baseline, and high GPU-share scenarios for 2013/14–2020/21. These percentages are modelling assumptions rather than observed historical shares.

The provisional conversion factor of 36 SU per GPU-hour is used as a common scenario proxy. It is not a verified conversion factor for Fornax or Topaz.

```
# Step 1: Historical GPU assumptions

historical_gpu_scenarios <- tibble::tribble(
  ~year, ~gpu_system_proxy, ~gpu_factor_proxy,
  ~gpu_share_low, ~gpu_share_baseline, ~gpu_share_high,

  2014, "Fornax",         36, 0.005, 0.010, 0.020,
```

```
  2015, "Fornax",          36, 0.005, 0.012, 0.025,
  2016, "Fornax",          36, 0.007, 0.015, 0.030,
  2017, "Fornax",          36, 0.008, 0.018, 0.035,
  2018, "Fornax",          36, 0.010, 0.020, 0.040,
  2019, "Fornax / Topaz", 36, 0.010, 0.025, 0.045,
  2020, "Gadi",            36, 0.015, 0.030, 0.050,
  2021, "Gadi",            36, 0.020, 0.035, 0.060
)

# Step 2: Calculate historical GPU estimates

historical_gpu_estimates <- conversion_data |>
  dplyr::filter(year %in% 2014:2021) |>
  dplyr::select(
    year,
    source_period,
    demand_ksu,
    allocation_ksu
  ) |>
  dplyr::left_join(
    historical_gpu_scenarios,
    by = "year"
  ) |>
  dplyr::mutate(
    allocation_gpu_ksu_low =
      allocation_ksu * gpu_share_low,

    allocation_gpu_ksu_baseline =
      allocation_ksu * gpu_share_baseline,

    allocation_gpu_ksu_high =
      allocation_ksu * gpu_share_high,

    demand_gpu_ksu_low =
      demand_ksu * gpu_share_low,

    demand_gpu_ksu_baseline =
      demand_ksu * gpu_share_baseline,

    demand_gpu_ksu_high =
      demand_ksu * gpu_share_high,

    allocation_gpu_hours_low =
      allocation_gpu_ksu_low * 1000 / gpu_factor_proxy,
```

```
    allocation_gpu_hours_baseline =
      allocation_gpu_ksu_baseline * 1000 / gpu_factor_proxy,

    allocation_gpu_hours_high =
      allocation_gpu_ksu_high * 1000 / gpu_factor_proxy,

    demand_gpu_hours_low =
      demand_gpu_ksu_low * 1000 / gpu_factor_proxy,

    demand_gpu_hours_baseline =
      demand_gpu_ksu_baseline * 1000 / gpu_factor_proxy,

    demand_gpu_hours_high =
      demand_gpu_ksu_high * 1000 / gpu_factor_proxy
  )

# Step 3: Validate historical estimates

stopifnot(
  nrow(historical_gpu_estimates) == 8,

  all(
    historical_gpu_estimates$gpu_share_low <=
      historical_gpu_estimates$gpu_share_baseline
  ),

  all(
    historical_gpu_estimates$gpu_share_baseline <=
      historical_gpu_estimates$gpu_share_high
  ),

  all(
    historical_gpu_estimates$allocation_gpu_ksu_high <=
      historical_gpu_estimates$allocation_ksu
  ),

  all(
    historical_gpu_estimates$demand_gpu_ksu_high <=
      historical_gpu_estimates$demand_ksu
  )
)

# Step 4: Display baseline results

historical_gpu_estimates |>
```

```
  dplyr::select(
    year,
    source_period,
    gpu_system_proxy,
    gpu_share_baseline,
    allocation_gpu_hours_baseline,
    demand_gpu_hours_baseline
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 8 x 6
   year source_period gpu_system_proxy gpu_share_baseline
  <dbl> <chr>         <chr>                         <dbl>
1  2014 2013/14       Fornax                        0.01
2  2015 2014/15       Fornax                        0.012
3  2016 2015/16       Fornax                        0.015
4  2017 2016/17       Fornax                        0.018
5  2018 2017/18       Fornax                        0.02
6  2019 2018/19       Fornax / Topaz                0.025
7  2020 2019/20       Gadi                          0.03
8  2021 2020/21       Gadi                          0.035
  allocation_gpu_hours_baseline demand_gpu_hours_baseline
                          <dbl>                     <dbl>
1                        23196.                    36488.
2                        29577                     67500
3                        40463.                    93750
4                        50000                    100000
5                       123333.                   293651.
6                       152778.                   458333.
7                       212623.                   506650
8                       386883.                  1160650.
```

```
# Reconcile historical CPU and GPU service units
# Baseline scenario only

historical_resource_reconciliation <- historical_gpu_estimates |>
  dplyr::mutate(
    allocation_cpu_ksu_baseline =
      allocation_ksu - allocation_gpu_ksu_baseline,

    demand_cpu_ksu_baseline =
      demand_ksu - demand_gpu_ksu_baseline,

    allocation_reconciliation =
      allocation_cpu_ksu_baseline +
      allocation_gpu_ksu_baseline -
```

```
      allocation_ksu,

    demand_reconciliation =
      demand_cpu_ksu_baseline +
      demand_gpu_ksu_baseline -
      demand_ksu
  )

# Validate reconciliation
stopifnot(
  all(abs(
    historical_resource_reconciliation$allocation_reconciliation
  ) < 1e-6),

  all(abs(
    historical_resource_reconciliation$demand_reconciliation
  ) < 1e-6)
)

# Display results
historical_resource_reconciliation |>
  dplyr::select(
    year,
    source_period,
    allocation_cpu_ksu_baseline,
    allocation_gpu_ksu_baseline,
    demand_cpu_ksu_baseline,
    demand_gpu_ksu_baseline,
    allocation_reconciliation,
    demand_reconciliation
  ) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 8 x 8
   year source_period allocation_cpu_ksu_baseline allocation_gpu_ksu_baseline
  <dbl> <chr>                              <dbl>                        <dbl>
1  2014 2013/14                           82670.                        835.
2  2015 2014/15                           87666.                       1065.
3  2016 2015/16                           95655.                       1457.
4  2017 2016/17                           98200                        1800
5  2018 2017/18                          217560                        4440
6  2019 2018/19                          214500                        5500
7  2020 2019/20                          247494.                       7654.
8  2021 2020/21                          384009.                      13928.
  demand_cpu_ksu_baseline demand_gpu_ksu_baseline allocation_reconciliation
```

```
                  <dbl>                  <dbl>                  <dbl>
1                130042.                 1314.                      0
2                200070                  2430                       0
3                221625                  3375                       0
4                196400                  3600                       0
5                518000.                10571.                      0
6                643500                 16500                       0
7                589741.                18239.                      0
8               1152028.                41783.                      0
  demand_reconciliation
                  <dbl>
1                     0
2                     0
3                     0
4                     0
5                     0
6                     0
7                     0
8                     0
```

```r
# Adjust historical CPU core-hours after separating GPU resources
# Preserve original CPU-equivalent estimates

historical_cpu_adjusted <- historical_resource_reconciliation |>
  dplyr::left_join(
    conversion_data |>
      dplyr::select(
        year,
        original_demand_cpu_core_hours = demand_cpu_core_hours,
        original_allocation_cpu_core_hours = allocation_cpu_core_hours
      ),
    by = "year"
  ) |>
  dplyr::mutate(
    # Preserve each year's original implied CPU conversion rate
    demand_cpu_core_hours_baseline =
      original_demand_cpu_core_hours *
      (demand_cpu_ksu_baseline / demand_ksu),

    allocation_cpu_core_hours_baseline =
      original_allocation_cpu_core_hours *
      (allocation_cpu_ksu_baseline / allocation_ksu)
  )

# Check results
historical_cpu_adjusted |>
```

```
  dplyr::select(
    year,
    source_period,
    original_demand_cpu_core_hours,
    demand_cpu_core_hours_baseline,
    original_allocation_cpu_core_hours,
    allocation_cpu_core_hours_baseline
  ) |>
  print(n = Inf)
```

```
# A tibble: 8 x 6
   year source_period original_demand_cpu_core_hours demand_cpu_core_hours_bas~1
  <dbl> <chr>                                  <dbl>                        <dbl>
1  2014 2013/14                           131356000                    130042440
2  2015 2014/15                           202500000                    200070000
3  2016 2015/16                           225000000                    221625000
4  2017 2016/17                           200000000                    196400000
5  2018 2017/18                           528571429.                   518000000.
6  2019 2018/19                           660000000                    643500000
7  2020 2019/20                           303990000                    294870300
8  2021 2020/21                           763191000                    736479315
# i abbreviated name: 1: demand_cpu_core_hours_baseline
# i 2 more variables: original_allocation_cpu_core_hours <dbl>,
#   allocation_cpu_core_hours_baseline <dbl>
```

```
# Validate adjusted CPU values
stopifnot(
  nrow(historical_cpu_adjusted) == 8,
  all(
    historical_cpu_adjusted$demand_cpu_core_hours_baseline <=
      historical_cpu_adjusted$original_demand_cpu_core_hours
  ),
  all(
    historical_cpu_adjusted$allocation_cpu_core_hours_baseline <=
      historical_cpu_adjusted$original_allocation_cpu_core_hours
  )
)
```

```
# Integrate historical CPU/GPU baseline estimates
# Preserve original CPU-equivalent values

historical_resource_baseline <- historical_cpu_adjusted |>
  dplyr::transmute(
    year,
    historical_demand_cpu_core_hours =
      demand_cpu_core_hours_baseline,
```

```
    historical_allocation_cpu_core_hours =
      allocation_cpu_core_hours_baseline,
    historical_demand_gpu_hours =
      demand_gpu_hours_baseline,
    historical_allocation_gpu_hours =
      allocation_gpu_hours_baseline,
    historical_demand_gpu_ksu =
      demand_gpu_ksu_baseline,
    historical_allocation_gpu_ksu =
      allocation_gpu_ksu_baseline,
    historical_demand_cpu_ksu =
      demand_cpu_ksu_baseline,
    historical_allocation_cpu_ksu =
      allocation_cpu_ksu_baseline
  )

# Preserve original CPU-equivalent values
conversion_data <- conversion_data |>
  dplyr::mutate(
    original_demand_cpu_core_hours = demand_cpu_core_hours,
    original_allocation_cpu_core_hours = allocation_cpu_core_hours
  ) |>
  dplyr::left_join(
    historical_resource_baseline,
    by = "year"
  ) |>
  dplyr::mutate(
    demand_cpu_core_hours = dplyr::coalesce(
      historical_demand_cpu_core_hours,
      demand_cpu_core_hours
    ),
    allocation_cpu_core_hours = dplyr::coalesce(
      historical_allocation_cpu_core_hours,
      allocation_cpu_core_hours
    ),
    demand_gpu_hours = dplyr::coalesce(
      historical_demand_gpu_hours,
      demand_gpu_hours
    ),
    allocation_gpu_hours = dplyr::coalesce(
      historical_allocation_gpu_hours,
      allocation_gpu_hours
    )
  ) |>
  dplyr::select(
```

```
    -dplyr::starts_with("historical_")
  )

# Validate complete annual CPU/GPU baseline
stopifnot(
  nrow(conversion_data) == 13,
  all(!is.na(conversion_data$demand_cpu_core_hours)),
  all(!is.na(conversion_data$allocation_cpu_core_hours)),
  all(!is.na(conversion_data$demand_gpu_hours)),
  all(!is.na(conversion_data$allocation_gpu_hours))
)

conversion_data |>
  dplyr::select(
    year,
    source_period,
    demand_cpu_core_hours,
    allocation_cpu_core_hours,
    demand_gpu_hours,
    allocation_gpu_hours
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 6
    year source_period demand_cpu_core_hours allocation_cpu_core_hours
   <dbl> <chr>                         <dbl>                     <dbl>
 1  2014 2013/14                   130042440                  82669950
 2  2015 2014/15                   200070000                  87666228
 3  2016 2015/16                   221625000                  95655320
 4  2017 2016/17                   196400000                  98200000
 5  2018 2017/18                   518000000.                217560000
 6  2019 2018/19                   643500000                 214500000
 7  2020 2019/20                   294870300                 123746780
 8  2021 2020/21                   736479315                 245493105
 9  2022 2021/22                   521210000                 260605000
10  2023 2022/23                   775466000                 491478000
11  2024 2023/24                   863240351.                492047000
12  2025 2024/25                   942053571.                527550000
13  2026 2025/26                  1235036000                 518644000
# i 2 more variables: demand_gpu_hours <dbl>, allocation_gpu_hours <dbl>
```

7.5 Field of Research Conversion

The Field of Research (FoR) conversion estimates the distribution of computational resources across research disciplines using the standardised FoR allocation

dataset from Chapter 04.

The conversion uses each field's annual allocation share to distribute the corresponding annual CPU core-hours and GPU-hours. This assumes that the resource composition within each field is proportional to the overall annual allocation composition.

The resulting FoR-level CPU and GPU values are estimates rather than directly observed resource-specific allocations. Original FoR observations and classification variables are retained to support traceability and subsequent analysis.

The 2016/17 reporting period requires particular caution because the available FoR information reflects Pawsey-wide reporting rather than the same NCMAS allocation scope used in other years. This difference will be explicitly flagged.

Before conversion, the FoR allocation shares are checked for completeness and consistency within each reporting year.

```
# Load standardised Field of Research dataset from Chapter 04

for_standardised <- readRDS(
  "assets/data/data standardised/field_of_research.rds"
)

# Inspect available variables
names(for_standardised)
```

```
 [1] "year"                                "source_period"
 [3] "research_field_reported"             "for_2008_division_code"
 [5] "for_2020_division_code"              "for_standard_2020"
 [7] "composition_pct"                     "allocation_share_for"
 [9] "requested_compute_annual"            "allocated_compute_annual"
[11] "annual_compute_unit"                 "estimated_demand_by_for_proxy"
[13] "estimated_allocated_compute_by_for" "requested_basis"
[15] "allocated_basis"                     "analytical_use"
[17] "scope_alignment"                     "evidence_quality"
[19] "scope"                               "value_basis"
[21] "classification_status"               "standardisation_note"
[23] "notes"                               "source"
[25] "source_url"                          "for_analysis_group"
```

```
# Inspect the structure
dplyr::glimpse(for_standardised)
```

```
Rows: 133
Columns: 26
$ year                              <dbl> 2014, 2014, 2014, 2014, 2014, 2014,~
$ source_period                     <chr> "2013/14", "2013/14", "2013/14", "2~
$ research_field_reported           <chr> "Earth & Environmental Sciences", "~
```

```
$ for_2008_division_code             <chr> "04 + 05", "02", "03", "09 + 10", "~
$ for_2020_division_code             <chr> "37 + 41", "51", "34", "40 + unmapp~
$ for_standard_2020                  <chr> "Earth sciences + Environmental sci~
$ composition_pct                    <dbl> 42.00, 22.00, 17.00, 11.00, 4.00, 3~
$ allocation_share_for               <dbl> 0.4200, 0.2200, 0.1700, 0.1100, 0.0~
$ requested_compute_annual           <dbl> 131356.0, 131356.0, 131356.0, 13135~
$ allocated_compute_annual           <dbl> 83505, 83505, 83505, 83505, 83505, ~
$ annual_compute_unit                <chr> "kSU", "kSU", "kSU", "kSU", "kSU", ~
$ estimated_demand_by_for_proxy      <dbl> 55169.520, 28898.320, 22330.520, 14~
$ estimated_allocated_compute_by_for <dbl> 35072.100, 18371.100, 14195.850, 91~
$ requested_basis                    <chr> "Direct", "Direct", "Direct", "Dire~
$ allocated_basis                    <chr> "Direct", "Direct", "Direct", "Dire~
$ analytical_use                     <chr> "Historical / lower confidence", "H~
$ scope_alignment                    <chr> "Historical / approximate", "Histor~
$ evidence_quality                   <chr> "Direct percentage / scope uncertai~
$ scope                              <chr> "NCI compute allocation; exact NCMA~
$ value_basis                        <chr> "Allocation composition", "Allocati~
$ classification_status              <chr> "As reported; FoR 2008/2020 alignme~
$ standardisation_note               <chr> "Historical combined category; cann~
$ notes                              <chr> "Percentage retained exactly as pub~
$ source                             <chr> "NCI Annual Report 2013/14", "NCI A~
$ source_url                         <chr> "https://nci.org.au/sites/default/f~
$ for_analysis_group                 <chr> "Earth & Environmental Sciences", "~
```

```r
# Check allocation shares by financial year
for_share_check <- for_standardised |>
  dplyr::group_by(year) |>
  dplyr::summarise(
    n_fields = dplyr::n(),

    total_allocation_share = sum(
      allocation_share_for,
      na.rm = TRUE
    ),

    missing_shares = sum(
      is.na(allocation_share_for)
    ),

    .groups = "drop"
  )

# Display all reporting years
for_share_check |>
  print(n = Inf)
```

```
# A tibble: 13 x 4
    year n_fields total_allocation_share missing_shares
   <dbl>    <int>                  <dbl>          <int>
 1  2014        7                  1                  0
 2  2015        5                  1                  0
 3  2016        7                  1                  0
 4  2017       17                  1                  0
 5  2018        8                  1                  0
 6  2019        9                  1                  0
 7  2020        8                  1                  0
 8  2021        9                  1                  0
 9  2022       13                  1.00               0
10  2023       12                  1.000              0
11  2024       17                  1                  0
12  2025       12                  1.000              0
13  2026        9                  1.000              0
```

```
# Validate that the annual dataset and FoR dataset
# use consistent analytical years
stopifnot(
  all(for_standardised$year %in% conversion_data$year),
  nrow(conversion_data) == 13
)
```

```
# Convert Field of Research allocations
# Preserve all original FoR observations and variables

for_converted <- for_standardised |>
  dplyr::left_join(
    conversion_data |>
      dplyr::select(
        year,
        annual_allocation_ksu = allocation_ksu,
        annual_allocation_cpu_core_hours =
          allocation_cpu_core_hours,
        annual_allocation_gpu_hours =
          allocation_gpu_hours
      ),
    by = "year"
  ) |>
  dplyr::mutate(
    # FoR allocation expressed in comparable annual kSU
    allocation_for_ksu =
      allocation_share_for * annual_allocation_ksu,

    # Estimated CPU allocation by research field
```

```
    allocation_for_cpu_core_hours =
      allocation_share_for *
      annual_allocation_cpu_core_hours,

    # Estimated GPU allocation by research field
    allocation_for_gpu_hours =
      allocation_share_for *
      annual_allocation_gpu_hours,

    # Record estimation method
    for_resource_conversion_method =
      "Proportional allocation using annual FoR share",

    # Flag reporting scope differences
    for_resource_scope_flag = dplyr::case_when(
      year == 2017 ~
        "Pawsey-wide FoR scope; not directly comparable to NCMAS",
      TRUE ~
        "Annual resource allocation distributed using FoR shares"
    ),

    # Distinguish historical scenario estimates
    for_resource_confidence = dplyr::case_when(
      year <= 2021 ~
        "Low: historical GPU scenario and proportional allocation",
      TRUE ~
        "Estimated: proportional CPU/GPU resource distribution"
    )
  )

# Check converted FoR results
for_converted |>
  dplyr::select(
    year,
    for_analysis_group,
    allocation_share_for,
    allocation_for_ksu,
    allocation_for_cpu_core_hours,
    allocation_for_gpu_hours
  ) |>
  print(n = 20, width = Inf)
```

```
# A tibble: 133 x 6
    year for_analysis_group             allocation_share_for allocation_for_ksu
   <dbl> <chr>                                         <dbl>              <dbl>
```

```
 1  2014 Earth & Environmental Sciences               0.42              35072.
 2  2014 Physical Sciences                            0.22              18371.
 3  2014 Chemical Sciences                            0.17              14196.
 4  2014 Engineering & Technology                     0.11               9186.
 5  2014 Biological Sciences                          0.04               3340.
 6  2014 Other / Unclassified                         0.03               2505.
 7  2014 Mathematical Sciences                        0.01                835.
 8  2015 Earth & Environmental Sciences               0.39              34605.
 9  2015 Physical Sciences                            0.23              20408.
10  2015 Chemical Sciences                            0.21              18634.
11  2015 Engineering & Technology                     0.13              11535.
12  2015 Biological Sciences                          0.04               3549.
13  2016 Earth & Environmental Sciences               0.302             29328.
14  2016 Mathematical Sciences                        0.235             22821.
15  2016 Chemical Sciences                            0.197             19131.
16  2016 Engineering & Technology                     0.165             16023.
17  2016 Biological Sciences                          0.039              3787.
18  2016 Physical Sciences                            0.032              3108.
19  2016 Other / Unclassified                         0.03               2913.
20  2017 Engineering & Technology                     0.305             30540
   allocation_for_cpu_core_hours allocation_for_gpu_hours
                           <dbl>                   <dbl>
 1                      34721379                    9742.
 2                      18187389                    5103.
 3                      14053892.                   3943.
 4                       9093694.                   2552.
 5                       3306798                     928.
 6                       2480098.                    696.
 7                        826700.                    232.
 8                      34189829.                  11535.
 9                      20163232.                   6803.
10                      18409908.                   6211.
11                      11396610.                   3845.
12                       3506649.                   1183.
13                      28887907.                  12220.
14                      22479000.                   9509.
15                      18844098.                   7971.
16                      15783128.                   6676.
17                       3730557.                   1578.
18                       3060970.                   1295.
19                       2869660.                   1214.
20                      29990280                   15270
# i 113 more rows
```

```
# Validate original FoR row count
stopifnot(
```

```
  nrow(for_converted) == nrow(for_standardised),
  nrow(for_converted) == 133
)
```

```
# Reconcile FoR allocations with annual resource totals

for_reconciliation <- for_converted |>
  dplyr::group_by(year) |>
  dplyr::summarise(
    for_total_ksu = sum(allocation_for_ksu),
    for_total_cpu_core_hours =
      sum(allocation_for_cpu_core_hours),
    for_total_gpu_hours =
      sum(allocation_for_gpu_hours),
    .groups = "drop"
  ) |>
  dplyr::left_join(
    conversion_data |>
      dplyr::select(
        year,
        allocation_ksu,
        allocation_cpu_core_hours,
        allocation_gpu_hours
      ),
    by = "year"
  ) |>
  dplyr::mutate(
    ksu_difference =
      for_total_ksu - allocation_ksu,

    cpu_difference =
      for_total_cpu_core_hours -
      allocation_cpu_core_hours,

    gpu_difference =
      for_total_gpu_hours -
      allocation_gpu_hours
  )

# Display reconciliation results
for_reconciliation |>
  dplyr::select(
    year,
    ksu_difference,
    cpu_difference,
```

```
    gpu_difference
  ) |>
  print(n = Inf)
```

```
# A tibble: 13 x 4
    year ksu_difference cpu_difference gpu_difference
   <dbl>          <dbl>          <dbl>          <dbl>
 1  2014     0                 0            0
 2  2015     0                 0            0
 3  2016     0                 0            0
 4  2017     0                 0            0
 5  2018     0                 0            0
 6  2019     0                 0            0
 7  2020     0                 0            0
 8  2021     0                 0            0
 9  2022     0.00000581        0.00261      0.0000133
10  2023    -0.0000245        -0.0147      -0.0000381
11  2024     0                 0            0
12  2025    -0.0000526        -0.0317      -0.0000802
13  2026    -0.0000785        -0.0467      -0.000113
```

```
# Validate differences using a relative tolerance
stopifnot(
  all(abs(for_reconciliation$ksu_difference) <
        pmax(1, abs(for_reconciliation$allocation_ksu)) * 1e-6),

  all(abs(for_reconciliation$cpu_difference) <
        pmax(1, abs(for_reconciliation$allocation_cpu_core_hours)) * 1e-6),

  all(abs(for_reconciliation$gpu_difference) <
        pmax(1, abs(for_reconciliation$allocation_gpu_hours)) * 1e-6)
)
```

7.6 Output

The Data Conversion stage produces three reproducible datasets for subsequent analysis and forecasting.

The annual converted dataset contains 13 financial years of HPC demand and allocation observations, including converted CPU core-hours and GPU-hours. Original reported values, source references, and conversion assumptions are retained.

The Field of Research converted dataset contains 133 observations with estimated allocations in kSU, CPU core-hours, and GPU-hours, distributed according to the standardised annual FoR allocation shares. These resource-specific field

estimates are based on proportional allocation assumptions rather than directly observed CPU or GPU usage.

A separate historical GPU scenario dataset preserves low, baseline, and high estimates for 2013/14 – 2020/21 to support sensitivity analysis.

All converted datasets are stored separately from the standardised source datasets. Reconciliation checks confirm that the FoR allocations reproduce annual resource totals within numerical tolerance.

Historical conversion factors, estimated resource shares, differences in reporting scope, and assumptions regarding CPU/GPU composition remain important limitations for interpretation and forecasting.

```
# Create output directory
output_dir <- "assets/data/data converted"

dir.create(
  output_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

# Save converted annual demand and allocation dataset
saveRDS(
  conversion_data,
  file.path(
    output_dir,
    "annual_demand_allocation_converted.rds"
  )
)

# Save converted Field of Research dataset
saveRDS(
  for_converted,
  file.path(
    output_dir,
    "field_of_research_converted.rds"
  )
)

# Preserve historical GPU uncertainty scenarios separately
saveRDS(
  historical_gpu_estimates,
  file.path(
    output_dir,
    "historical_gpu_scenarios.rds"
```

```
  )
)

# Verify that all files were created
output_files <- file.path(
  output_dir,
  c(
    "annual_demand_allocation_converted.rds",
    "field_of_research_converted.rds",
    "historical_gpu_scenarios.rds"
  )
)

stopifnot(all(file.exists(output_files)))

# Verify exported dataset dimensions
stopifnot(
  nrow(readRDS(output_files[1])) == 13,
  nrow(readRDS(output_files[2])) == 133,
  nrow(readRDS(output_files[3])) == 8
)

cat("All three converted datasets exported successfully.\n")
```

```
All three converted datasets exported successfully.
```

Chapter 8

Initial Analysis

8.1 Purpose

The purpose of this stage is to examine historical demand and allocation patterns for Australian national High-Performance Computing (HPC) resources from 2014 to 2026, using the converted datasets produced in Chapter 05.

The analysis focuses on identifying trends in computational resource demand, allocation levels, and potential differences between requested and allocated resources. CPU core-hours and GPU-hours are examined separately because they represent distinct computational resource types and cannot be directly combined.

The analysis also examines the distribution of HPC allocations across Fields of Research (FoR) to identify how allocated computational resources are distributed among research disciplines and how these patterns have changed over time.

Particular attention is given to historical reporting differences, conversion assumptions, and estimated resource values. These limitations are considered when interpreting trends and assessing the suitability of the data for forecasting.

The findings from this stage will inform subsequent forecasting model development, evaluation, and scenario analysis through 2030.

```
annual_converted <- readRDS(
  "assets/data/data converted/annual_demand_allocation_converted.rds"
)

for_converted <- readRDS(
  "assets/data/data converted/field_of_research_converted.rds"
)
```

```
historical_gpu_scenarios <- readRDS(
  "assets/data/data converted/historical_gpu_scenarios.rds"
)
```

8.2 Historical Demand and Allocation

Historical HPC demand and allocation are examined across the 2014 - 2026 analytical period to identify changes in requested computational resources, awarded allocations, and the difference between them.

Requested compute is used as an indicator of researcher demand, while allocated compute reflects the resources awarded within available infrastructure and programme constraints. These measures are therefore interpreted separately.

The analysis uses the converted annual kSU series to examine demand and allocation trends, annual growth rates, request allocation gaps, and fulfilment rates. However, historical values include directly reported observations, derived estimates, and capacity-based proxies. Consequently, observed changes may partly reflect reporting differences or changes in infrastructure rather than underlying demand alone.

8.2.1 Annual Data Validation

```
library(dplyr)
library(ggplot2)

# Inspect the annual demand and allocation series
annual_converted |>
  select(
    year,
    demand_ksu,
    allocation_ksu,
    conversion_status,
    conversion_note
  ) |>
  arrange(year) |>
  print(n = Inf, width = Inf)
```

```
# A tibble: 13 x 5
    year demand_ksu allocation_ksu conversion_status
   <dbl>      <dbl>          <dbl> <chr>
 1  2014     131356          83505 Reported kSU; resource conversion assumed
 2  2015     202500          88731 Demand kSU estimated; allocation kSU reported
 3  2016     225000          97112 Demand kSU estimated; allocation kSU reported
 4  2017     200000         100000 Both kSU estimated; capacity proxy used
```

```
 5  2018    528571.         222000 Both kSU estimated; allocation reported
 6  2019    660000          220000 Both kSU estimated; capacity proxy used
 7  2020    607980          255148 Original kSU retained
 8  2021   1193811          397937 Original kSU retained
 9  2022   1162230          581115 Original kSU retained
10  2023   1333240          817956 Original kSU retained
11  2024   1425560.         812569 Original kSU retained
12  2025   1564643.         876200 Original kSU retained
13  2026   2242817          872288 Original kSU retained
   conversion_note
   <chr>
 1 Original reported kSU retained; CPU/GPU resource split not established.
 2 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU~
 3 Demand million hours x 1,000,000 x assumed 1 SU/hour / 1,000; allocation kSU~
 4 Million core-hours converted using assumed 1 SU/core-hour; capacity is an al~
 5 Million hours converted using assumed 1 SU/hour; original rounded values ret~
 6 Million CPU-hours converted using assumed 1 SU/CPU-hour; available capacity ~
 7 <NA>
 8 <NA>
 9 <NA>
10 <NA>
11 <NA>
12 <NA>
13 <NA>
```

```r
# Check coverage and completeness
stopifnot(
  nrow(annual_converted) == 13,
  all(sort(annual_converted$year) == 2014:2026),
  all(!is.na(annual_converted$demand_ksu)),
  all(!is.na(annual_converted$allocation_ksu)),
  all(annual_converted$demand_ksu > 0),
  all(annual_converted$allocation_ksu >= 0)
)
```

8.2.2 Annual Analytical Indicators

```r
annual_analysis <- annual_converted |>
  arrange(year) |>
  mutate(
    demand_allocation_gap_ksu =
      demand_ksu - allocation_ksu,

    fulfilment_rate_pct =
      100 * allocation_ksu / demand_ksu,
```

```
    demand_growth_pct =
      100 * (demand_ksu / lag(demand_ksu) - 1),

    allocation_growth_pct =
      100 * (allocation_ksu / lag(allocation_ksu) - 1)
  )

annual_analysis |>
  select(
    year,
    demand_ksu,
    allocation_ksu,
    demand_allocation_gap_ksu,
    fulfilment_rate_pct,
    demand_growth_pct,
    allocation_growth_pct
  ) |>
  print(n = Inf)
```

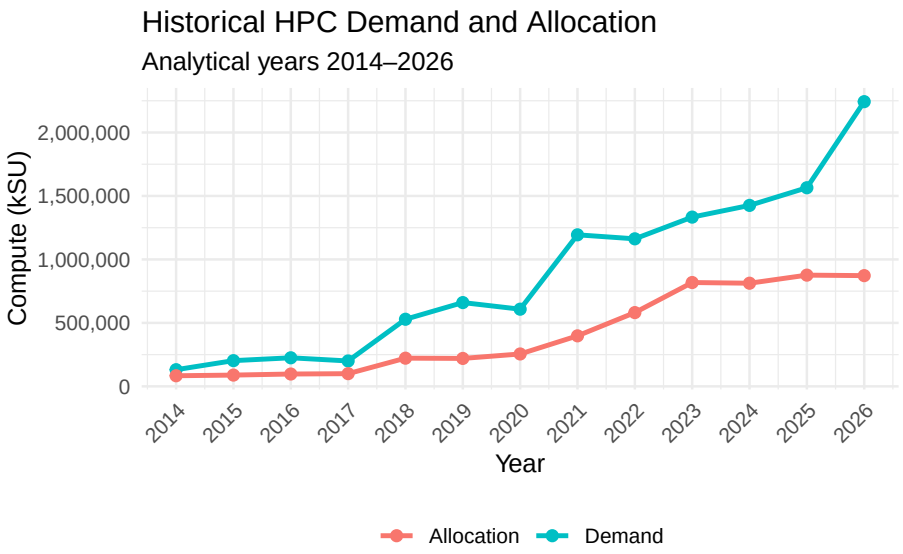
```
# A tibble: 13 x 7
    year demand_ksu allocation_ksu demand_allocation_gap_ksu fulfilment_rate_pct
   <dbl>      <dbl>          <dbl>                     <dbl>               <dbl>
 1  2014     131356          83505                     47851                63.6
 2  2015     202500          88731                    113769                43.8
 3  2016     225000          97112                    127888                43.2
 4  2017     200000         100000                    100000                50
 5  2018     528571.        222000                    306571.               42.0
 6  2019     660000         220000                    440000                33.3
 7  2020     607980         255148                    352832                42.0
 8  2021    1193811         397937                    795874                33.3
 9  2022    1162230         581115                    581115                50
10  2023    1333240         817956                    515284                61.4
11  2024    1425560.        812569                    612991.               57
12  2025    1564643.        876200                    688443.               56
13  2026    2242817         872288                   1370529                38.9
# i 2 more variables: demand_growth_pct <dbl>, allocation_growth_pct <dbl>
```

8.2.3 Annual Demand and Allocation Trends

```
annual_plot_data <- annual_analysis |>
  select(
    year,
    Demand = demand_ksu,
    Allocation = allocation_ksu
  ) |>
```

```
  tidyr::pivot_longer(
    cols = c(Demand, Allocation),
    names_to = "measure",
    values_to = "value_ksu"
  )

ggplot(
  annual_plot_data,
  aes(
    x = year,
    y = value_ksu,
    colour = measure
  )
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2014:2026) +
  scale_y_continuous(
    labels = scales::label_comma()
  ) +
  labs(
    title = "Historical HPC Demand and Allocation",
    subtitle = "Analytical years 2014-2026",
    x = "Year",
    y = "Compute (kSU)",
    colour = NULL
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )
```



8.2.4 Initial Findings

The annual HPC demand and allocation series show an overall upward trend between 2014 and 2026. Reported or estimated demand increases from approximately 131,356 kSU in 2014 to 2,242,817 kSU in 2026, while allocation increases from approximately 83,505 kSU to 872,288 kSU.

Demand exceeds allocation in every analytical year. The largest absolute gap occurs in 2026, when demand exceeds allocation by approximately 1.37 million kSU. The corresponding fulfilment rate is approximately 38.9%, compared with 56.0% in 2025.

The series also exhibits periods of substantial change, particularly around 2018, 2021, and 2026. However, these changes should not automatically be interpreted as equivalent increases in underlying researcher demand. Historical conversion assumptions, differences in reporting scope, capacity-based allocation proxies, and changes in HPC systems may contribute to the observed variation.

These findings suggest a persistent difference between requested and allocated computational resources. Further analysis of CPU and GPU resources is required to understand whether similar patterns are present across distinct computational resource types.

8.3 CPU and GPU Resources

CPU and GPU resources are examined separately to identify historical patterns in computational demand and allocation across the 2014–2026 analytical period.

The analysis uses the converted CPU core-hours and GPU-hours produced in Chapter 05. These measures provide resource-specific perspectives on HPC demand and allocation, although their interpretation remains subject to system-level accounting conventions and historical conversion assumptions.

Historical CPU estimates are influenced by the assumed resource composition and conversion factors applied to earlier observations. GPU estimates for 2014 - 2021 are based on low, baseline, and high scenarios rather than directly observed resource-level demand and allocation. GPU values for 2022 - 2026 use reported system-level allocations and a combination of reported and estimated demand.

CPU core-hours and GPU-hours are therefore analysed independently and are not added together. The objective is to examine trends within each resource category while retaining the distinction between reported observations and modelled estimates.

8.3.1 Validate the converted CPU and GPU series

```
# Inspect converted CPU and GPU resources
annual_converted |>
  select(
    year,
    demand_cpu_core_hours,
    allocation_cpu_core_hours,
    demand_gpu_hours,
    allocation_gpu_hours
  ) |>
  arrange(year) |>
  print(n = Inf)
```

```
# A tibble: 13 x 5
    year demand_cpu_core_hours allocation_cpu_core_hours demand_gpu_hours
   <dbl>                 <dbl>                     <dbl>            <dbl>
 1  2014             130042440                  82669950           36488.
 2  2015             200070000                  87666228           67500
 3  2016             221625000                  95655320           93750
 4  2017             196400000                  98200000          100000
 5  2018             518000000.                217560000          293651.
 6  2019             643500000                 214500000          458333.
 7  2020             294870300                 123746780          506650
 8  2021             736479315                 245493105         1160650.
 9  2022             521210000                 260605000         2663281.
10  2023             775466000                 491478000         1346414.
11  2024             863240351.                492047000         2297560.
12  2025             942053571.                527550000         2386300.
13  2026            1235036000                 518644000         3003281.
```

```
# i 1 more variable: allocation_gpu_hours <dbl>
```

```
# Check completeness and non-negative values
resource_columns <- c(
  "demand_cpu_core_hours",
  "allocation_cpu_core_hours",
  "demand_gpu_hours",
  "allocation_gpu_hours"
)

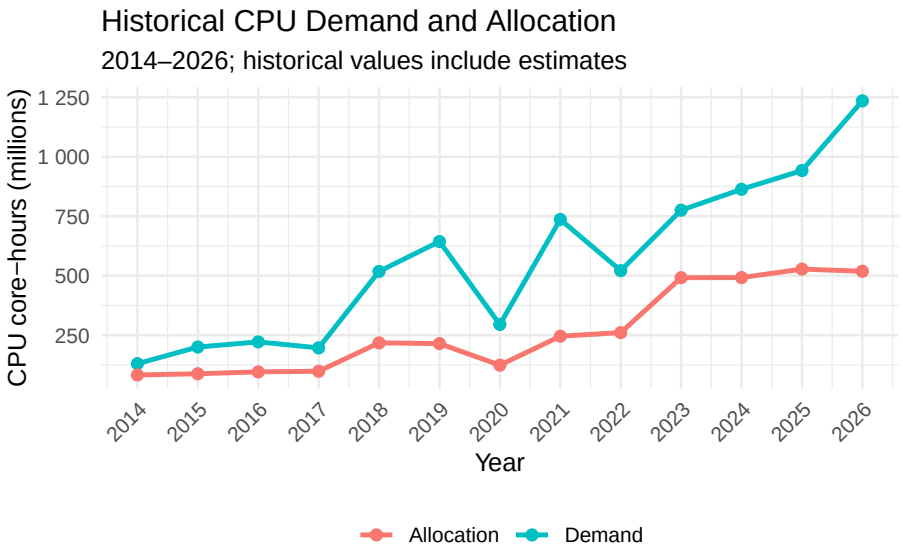
stopifnot(
  nrow(annual_converted) == 13,
  all(sort(annual_converted$year) == 2014:2026),
  all(!is.na(annual_converted[resource_columns])),
  all(as.matrix(annual_converted[resource_columns]) >= 0)
)
```

8.3.2 Create CPU and GPU trend charts

```
# CPU demand and allocation trends
cpu_plot_data <- annual_converted |>
  select(
    year,
    Demand = demand_cpu_core_hours,
    Allocation = allocation_cpu_core_hours
  ) |>
  tidyr::pivot_longer(
    cols = c(Demand, Allocation),
    names_to = "measure",
    values_to = "cpu_core_hours"
  )

ggplot(
  cpu_plot_data,
  aes(
    x = year,
    y = cpu_core_hours,
    colour = measure
  )
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2014:2026) +
  scale_y_continuous(
    labels = scales::label_number(
      scale = 1e-6,
```

```
      accuracy = 1
    )
  ) +
  labs(
    title = "Historical CPU Demand and Allocation",
    subtitle = "2014–2026; historical values include estimates",
    x = "Year",
    y = "CPU core-hours (millions)",
    colour = NULL
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )
```



The CPU demand and allocation trends represent converted baseline estimates. Historical CPU values are subject to uncertainty arising from system-specific accounting conventions, assumed CPU/GPU resource composition, and the use of capacity proxies in selected years. Unlike the historical GPU analysis, separate low and high CPU core-hour scenarios have not been established. Therefore, the CPU chart presents the baseline series without uncertainty bands, and historical fluctuations should be interpreted cautiously.

```
# Prepare GPU demand and allocation data
gpu_plot_data <- annual_converted |>
  dplyr::select(
```

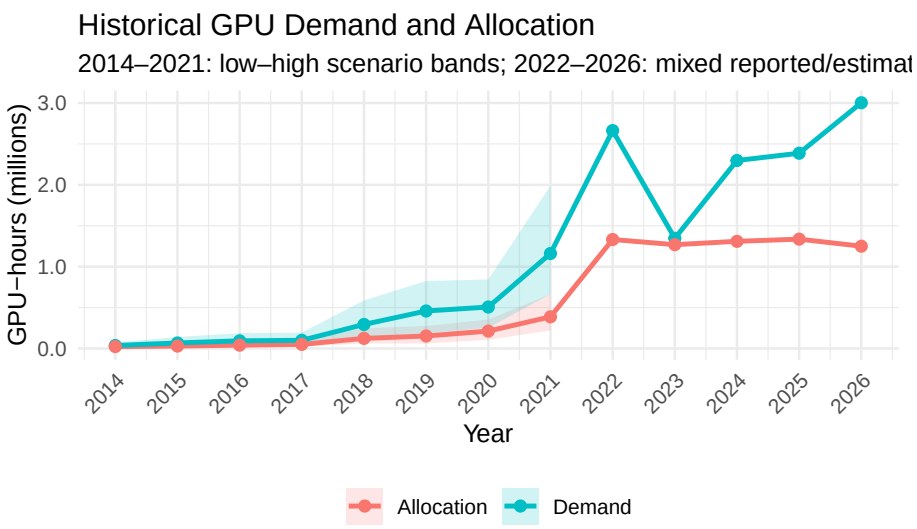
```
    year,
    Demand = demand_gpu_hours,
    Allocation = allocation_gpu_hours
  ) |>
  tidyr::pivot_longer(
    cols = c(Demand, Allocation),
    names_to = "measure",
    values_to = "gpu_hours"
  )

# Use the existing historical low/high GPU scenarios
gpu_uncertainty_data <- historical_gpu_scenarios |>
  dplyr::select(
    year,
    demand_gpu_hours_low,
    demand_gpu_hours_high,
    allocation_gpu_hours_low,
    allocation_gpu_hours_high
  ) |>
  tidyr::pivot_longer(
    cols = -year,
    names_to = c("measure", ".value"),
    names_pattern = "(demand|allocation)_(gpu_hours_low|gpu_hours_high)"
  ) |>
  dplyr::mutate(
    measure = dplyr::recode(
      measure,
      demand = "Demand",
      allocation = "Allocation"
    )
  )

# Plot baseline trends with historical uncertainty bands
ggplot() +
  geom_ribbon(
    data = gpu_uncertainty_data,
    aes(
      x = year,
      ymin = gpu_hours_low,
      ymax = gpu_hours_high,
      fill = measure,
      group = measure
    ),
    alpha = 0.18,
    colour = NA
```

```
  ) +
  geom_line(
    data = gpu_plot_data,
    aes(
      x = year,
      y = gpu_hours,
      colour = measure
    ),
    linewidth = 1
  ) +
  geom_point(
    data = gpu_plot_data,
    aes(
      x = year,
      y = gpu_hours,
      colour = measure
    ),
    size = 2
  ) +
  scale_x_continuous(
    breaks = 2014:2026
  ) +
  scale_y_continuous(
    labels = scales::label_number(
      scale = 1e-6,
      accuracy = 0.1
    )
  ) +
  labs(
    title = "Historical GPU Demand and Allocation",
    subtitle = "2014-2021: low-high scenario bands; 2022-2026: mixed reported/estimated",
    x = "Year",
    y = "GPU-hours (millions)",
    colour = NULL,
    fill = NULL,
    caption = paste(
      "Shaded bands represent assumed historical GPU scenarios,",
      "not statistical confidence intervals."
    )
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
```

```
    ),
    legend.position = "bottom"
  )
```



Shaded bands represent assumed historical GPU scenarios, not statistical confidence intervals.

The shaded bands represent the range between the low and high historical GPU estimation scenarios for 2014 - 2021, while the solid lines represent the baseline demand and allocation estimates. These bands illustrate sensitivity to assumed historical GPU resource shares rather than statistical confidence intervals. No equivalent uncertainty bands are shown for 2022 - 2026, although those observations remain subject to conversion assumptions and, for some years, estimated GPU demand.

```
historical_gpu_scenarios |>
  select(
    year,
    allocation_gpu_hours_low,
    allocation_gpu_hours_baseline,
    allocation_gpu_hours_high,
    demand_gpu_hours_low,
    demand_gpu_hours_baseline,
    demand_gpu_hours_high
  ) |>
  arrange(year) |>
  print(n = Inf)
```

```
# A tibble: 8 x 7
   year allocation_gpu_hours_low allocation_gpu_hours_b~1 allocation_gpu_hours~2
```

```
  <dbl>                  <dbl>                  <dbl>                  <dbl>
1  2014                 11598.                 23196.                 46392.
2  2015                 12324.                 29577                  61619.
3  2016                 18883.                 40463.                 80927.
4  2017                 22222.                 50000                  97222.
5  2018                 61667.                123333.                246667.
6  2019                 61111.                152778.                275000
7  2020                106312.                212623.                354372.
8  2021                221076.                386883.                663228.
# i abbreviated names: 1: allocation_gpu_hours_baseline,
#   2: allocation_gpu_hours_high
# i 3 more variables: demand_gpu_hours_low <dbl>,
#   demand_gpu_hours_baseline <dbl>, demand_gpu_hours_high <dbl>
```

8.3.3 Initial Findings

The resource-specific analysis indicates an overall increase in estimated CPU and GPU demand between 2014 and 2026, although the two series exhibit different patterns and levels of uncertainty.

CPU Resources

Estimated CPU demand increases from approximately 130.0 million CPU core-hours in 2014 to 1,235.0 million CPU core-hours in 2026. Over the same period, CPU allocation increases from approximately 82.7 million to 518.6 million CPU core-hours.

CPU demand remains above allocation throughout the analytical period. Substantial fluctuations are visible around 2018–2022, followed by a sustained increase in estimated demand from 2022 to 2026. In contrast, CPU allocation remains relatively stable between 2023 and 2026, at approximately 491–528 million CPU core-hours.

The widening difference between CPU demand and allocation in recent years suggests increasing pressure on allocated CPU resources. However, the magnitude of historical changes should be interpreted cautiously because earlier observations include assumed conversion factors, capacity proxies, and estimated CPU/GPU resource shares.

GPU Resources

Estimated GPU demand increases from approximately 0.04 million GPU-hours in 2014 to 3.00 million GPU-hours in 2026. GPU allocation increases from approximately 0.02 million to 1.25 million GPU-hours over the same period.

The GPU series shows a pronounced increase between 2021 and 2022, followed by fluctuations in estimated demand and comparatively stable allocations during 2022–2026. By 2026, estimated GPU demand is approximately 2.4 times the allocated GPU-hours.

However, the historical GPU series requires particular caution. Values for 2014–2021 are based on assumed GPU resource shares and a provisional conversion factor rather than directly observed GPU demand and allocation. The low, baseline, and high scenarios demonstrate that historical GPU estimates are sensitive to these assumptions.

Furthermore, GPU allocation from 2022 onward is derived from reported system-level service units using a working conversion factor, while GPU demand includes both directly reported and estimated values. The apparent increase around 2022 may therefore partly reflect changes in reporting coverage, infrastructure, and conversion methodology rather than an equivalent increase in underlying GPU demand.

Implications for Forecasting

The CPU and GPU series suggest different historical demand patterns and should be considered separately during forecasting model development. The observed differences between demand and allocation also reinforce the importance of distinguishing researcher demand from computational resources awarded under capacity constraints.

Historical uncertainty, system transitions, and the limited availability of directly observed GPU demand should be considered when selecting modelling periods and evaluating forecast reliability. Scenario analysis will be particularly important for assessing the sensitivity of GPU forecasts to historical conversion assumptions.

Chapter 9

Model Development and Benchmarking

9.1 Purpose

The purpose of this step is to develop forecasting models for Australian research HPC demand and compare their performance.

The modelling approach includes simple benchmark models as well as models that incorporate potential structural drivers of HPC demand.

Because the historical dataset is relatively short and includes changes in HPC systems, reporting methods and resource types, the models are kept relatively simple and interpretable.

9.2 Forecast Target

The primary forecasting target is Australian research demand for national HPC resources.

Where the available data support it, CPU and GPU demand are modelled separately because they represent different types of computational resources and have different historical growth patterns.

Allocated compute is treated separately from demand because allocation is constrained by available national HPC capacity.

9.3 Benchmark Models

Simple forecasting models are used to provide a baseline against which more complex models can be compared.

The benchmark models considered include:

- naïve
- drift
- linear trend
- exponential smoothing (ETS)
- ARIMA

These models provide different representations of the historical trend without requiring external explanatory variables.

9.4 Structural Demand Model

In addition to the benchmark models, a structural modelling approach is considered to explain HPC demand using potential demand drivers.

Potential drivers identified in the earlier stages include:

- research funding
- research workforce
- research activity
- funding in compute-intensive research fields
- compute intensity
- changes in CPU and GPU use

A general form of the structural model is:

$$D_t = \beta_0 + \beta_1 X_{1,t} + \beta_2 X_{2,t} + \cdots + \beta_k X_{k,t} + \varepsilon_t$$

where D_t represents HPC demand and $X_{1,t}, \ldots, X_{k,t}$ represent selected demand drivers.

9.5 Leading Indicators

Some structural variables may influence HPC demand with a delay.

For example, research funding may be considered as a leading indicator because funded research projects may require HPC resources after funding has been awarded.

Where supported by the data, lagged variables are therefore considered:

$$D_t = \beta_0 + \beta_1 Funding_{t-1} + \beta_2 Workforce_t + \beta_3 ComputeIntensity_t + \varepsilon_t$$

The appropriate lag structure is determined using the available data and the expected relationship between each driver and HPC demand.

9.6 Variable Selection

The number of predictors included in the structural model is kept small because the historical annual dataset contains a limited number of observations.

Variables are considered based on:

- theoretical relationship with HPC demand
- historical data availability
- consistency across years
- exploratory relationships with demand
- availability of future values or assumptions to 2030
- interpretability for policy analysis

Variables are not included solely because they improve in-sample model fit.

9.7 Model Benchmarking

The candidate models are compared using a consistent evaluation framework.

Where possible, rolling-origin time-series cross-validation is used so that models are evaluated using only information that would have been available at the time of each forecast.

Forecast accuracy measures considered include:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE), where appropriate

Forecast behaviour and interpretability are also considered alongside numerical accuracy.

9.8 CPU and GPU Modelling

Where sufficient historical information is available, CPU and GPU demand are modelled separately.

This is particularly important for GPU resources because growth in AI, machine learning and other accelerator-intensive research may result in a different demand trajectory from traditional CPU computing.

If the historical GPU series is insufficient for reliable standalone forecasting, alternative scenario-based assumptions are considered rather than fitting an unnecessarily complex statistical model.

9.9 Modelling Limitations

The modelling approach is affected by several limitations:

- a relatively short annual time series
- changes in HPC systems and accounting methods
- incomplete historical requested-demand information
- supply constraints affecting observed allocations
- limited historical GPU observations
- changes in research classifications
- some values being derived or estimated rather than directly reported

These limitations are considered when interpreting model performance and selecting the final forecasting approach.

9.10 Output

The output of this step is a set of candidate forecasting models and benchmark results.

These models are evaluated in the next step before the final forecasting approach is selected for scenario analysis to 2030.

Chapter 10

Model Evaluation

10.1 Purpose

The purpose of this step is to evaluate the forecasting models developed in the previous step and assess how well they represent historical HPC demand.

Model evaluation considers forecast accuracy, residual behaviour, forecast behaviour and interpretability. The aim is not only to identify models with low forecast errors, but also to ensure that the resulting forecasts are reasonable for the available data and the purpose of the project.

10.2 Evaluation Approach

The candidate models are evaluated using a consistent framework.

Where possible, time-series cross-validation is used to assess how the models would have performed when forecasting observations that were not used during model estimation.

This is particularly important because evaluating models only on their in-sample fit may give an overly optimistic view of forecasting performance.

10.3 Time-Series Cross-Validation

Rolling-origin time-series cross-validation is used where the available historical data allow it.

At each evaluation point, the model is estimated using observations available up to that point and then used to forecast future observations.

The forecasting origin is progressively moved forward through the historical period.

This approach maintains the chronological order of the data and avoids using future information to predict the past.

10.4 Forecast Accuracy

Forecast accuracy is assessed using measures such as:

10.4.1 Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$$

MAE measures the average absolute difference between observed and forecast values.

10.4.2 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$$

RMSE gives greater weight to larger forecast errors.

10.4.3 Mean Absolute Percentage Error

$$MAPE = \frac{100}{n}\sum_{t=1}^{n}\left|\frac{y_t - \hat{y}_t}{y_t}\right|$$

MAPE may be used as a supplementary measure where the observed values make percentage errors meaningful.

No single accuracy measure is used in isolation when comparing the models.

10.5 Residual Diagnostics

Residuals are examined to determine whether important patterns remain unexplained by the model.

The evaluation considers:

- residuals over time
- residual autocorrelation
- unusually large residuals
- systematic over- or under-forecasting

- residual distribution

A well-performing forecasting model should not leave strong systematic patterns in its residuals.

10.6 Forecast Behaviour

The resulting forecasts are also examined visually.

This is important because a model can achieve relatively good historical accuracy while producing unrealistic long-term forecasts.

The evaluation therefore considers whether:

- the direction of the forecast is reasonable
- long-term growth is plausible
- forecasts become unstable
- prediction intervals appropriately reflect uncertainty
- forecasts respond reasonably to the historical data

10.7 Structural Model Evaluation

For models containing structural demand drivers, evaluation also considers whether the estimated relationships are interpretable and consistent with the expected relationship between the variables and HPC demand.

Particular attention is given to:

- coefficient direction
- sensitivity to individual observations
- model complexity
- stability of estimated relationships
- contribution of the selected drivers

Given the limited number of annual observations, model complexity is considered carefully.

10.8 Comparison with Benchmark Models

The structural models are compared with the simpler benchmark models developed in Step 7.

A more complex model is not automatically preferred simply because it contains additional explanatory variables.

The additional complexity should provide useful forecasting performance or interpretation relative to simpler alternatives.

10.9 CPU and GPU Evaluation

Where CPU and GPU demand are modelled separately, their models are evaluated independently.

This allows the evaluation approach to reflect differences in historical data availability and demand patterns between the two resource types.

10.10 Evaluation Limitations

Model evaluation is constrained by the relatively short historical dataset.

As a result:

- the number of cross-validation periods may be limited
- accuracy measures may be sensitive to individual years
- structural changes may affect historical model performance
- GPU evaluation may be based on fewer observations than CPU evaluation

These limitations are considered when interpreting differences in model performance.

10.11 Output

The output of this step is a comparison of candidate models based on forecast accuracy, residual diagnostics, forecast behaviour and interpretability.

The evaluation results are then used to support model selection in the next step.

Chapter 11

Model Selection

11.1 Purpose

The purpose of this step is to select the modelling approach that will be used to forecast Australian research HPC demand to 2030.

The selection is based on the evaluation results from the previous step. Forecast accuracy is considered alongside model stability, interpretability, forecast behaviour and suitability for scenario analysis.

11.2 Selection Criteria

Candidate models are compared using several criteria:

- out-of-sample forecast accuracy
- residual diagnostics
- stability of the model
- plausibility of the resulting forecasts
- interpretability
- suitability for the available historical data
- ability to support forecasting to 2030
- suitability for scenario analysis

No model is selected based on a single accuracy measure.

11.3 Benchmark and Structural Models

The performance of the structural demand models is compared with the simpler benchmark models.

Benchmark models provide a useful reference for determining whether additional structural variables improve the forecasting approach.

Where a more complex model does not provide a meaningful improvement over a simpler model, the simpler model may be preferred.

11.4 Structural Driver Selection

For structural models, the final set of explanatory variables is selected based on both statistical performance and their relationship with HPC demand.

The selected variables should:

- have a clear relationship with HPC demand
- have sufficient historical coverage
- be reasonably consistent over time
- avoid unnecessary model complexity
- support assumptions or projections to 2030
- provide a useful interpretation for policy analysis

Given the limited number of annual observations, the final model is kept parsimonious.

11.5 CPU and GPU Modelling Approach

CPU and GPU resources are considered separately when selecting the final forecasting approach.

Different models may be selected for CPU and GPU demand if the historical data and demand patterns support different approaches.

Where the GPU time series is too limited for reliable statistical forecasting, a scenario-based approach may be used instead.

11.6 Expert Review

The modelling results and assumptions may also be reviewed with relevant HPC experts.

Expert feedback is used to assess whether the model assumptions, historical interpretation and resulting forecasts are reasonable in the context of Australian national research computing.

Any changes resulting from expert feedback are documented.

11.7 Final Model Selection

The final model or combination of models is selected after considering the quantitative evaluation results and the broader analytical requirements of the project.

The selected approach should provide a reasonable balance between forecasting performance, interpretability and robustness given the limitations of the historical data.

The rationale for the final selection will be documented together with the model specification and key assumptions.

11.8 Output

The output of this step is the selected forecasting approach for projecting Australian research HPC demand to 2030.

The selected model is then used as the basis for scenario forecasting and capacity-gap analysis in the final step.

Chapter 12

Scenario Forecasting and Capacity-Gap Analysis

12.1 Purpose

The purpose of this step is to use the selected modelling approach to forecast Australian research HPC demand to 2030 under different scenarios.

The scenarios are designed to represent different possible developments in Australian research activity and compute intensity rather than producing a single estimate of future demand.

The resulting demand forecasts are then compared with available or projected national HPC capacity to identify potential capacity gaps.

12.2 Forecast Horizon

The selected modelling approach is used to produce forecasts to 2030.

Forecast uncertainty is reported where appropriate to reflect the uncertainty associated with both the historical data and future assumptions.

12.3 Scenario Framework

Three main scenarios are considered:

1. Status quo
2. Restored national R&D intensity
3. AI acceleration

Each scenario uses the same overall modelling framework but applies different assumptions about the future development of relevant demand drivers.

12.4 Status Quo Scenario

The status quo scenario represents a continuation of current research and HPC demand conditions.

Future values of the structural drivers are based on recent trends or other reasonable baseline assumptions.

This scenario provides a baseline against which the alternative scenarios can be compared.

12.5 Restored National R&D Intensity Scenario

This scenario considers higher research activity associated with an increase in Australia's research and development intensity.

Changes in relevant drivers, such as research funding and research activity, are translated into future HPC demand using the selected modelling approach.

The assumptions used for this scenario are documented explicitly.

12.6 AI Acceleration Scenario

The AI acceleration scenario considers stronger growth in compute-intensive research associated with increasing use of artificial intelligence, machine learning and accelerated computing.

Particular attention is given to GPU demand because these workloads may have different resource requirements from traditional CPU-based research computing.

The scenario assumptions are based on available evidence and are documented separately from the historical observations.

12.7 CPU and GPU Forecasts

Where supported by the available data, CPU and GPU demand are forecast separately.

This avoids assuming that CPU core-hours and GPU hours are directly interchangeable and allows differences in their future growth to be examined.

12.8 Capacity Comparison

Forecast demand is compared with available or projected national HPC capacity where suitable capacity information is available.

Demand and capacity are only compared where their units and definitions are sufficiently consistent.

The general capacity-gap calculation is:

$$\text{Capacity Gap}_t = \text{Forecast Demand}_t - \text{Available Capacity}_t$$

A positive value indicates forecast demand above the capacity represented in the comparison.

12.9 Facility-Equivalent Capacity

Where sufficient information is available, the estimated capacity gap may also be expressed in facility-equivalent terms.

This provides a more interpretable indication of the scale of additional computing infrastructure that may be required.

Any facility-equivalent calculation depends on assumptions about system capacity, resource type and utilisation. These assumptions are therefore reported explicitly.

12.10 Scenario Comparison

The three scenarios are compared to examine how different assumptions affect projected HPC demand to 2030.

The comparison focuses on:

- total forecast demand
- CPU demand
- GPU demand
- growth relative to the historical period
- estimated capacity gaps
- differences between scenarios

12.11 Uncertainty and Sensitivity

Scenario forecasts are not treated as precise predictions of future HPC demand.

Results depend on the selected model, historical data, conversion assumptions and future values assumed for the structural drivers.

Sensitivity analysis is used where appropriate to examine how changes in key assumptions affect the forecast results.

12.12 Policy Interpretation

The scenario results are interpreted in the context of Australia's future national research computing requirements.

The analysis focuses on the potential scale and composition of future demand and the implications of different research and technology trajectories for national HPC infrastructure.

12.13 Output

The output of this step is a set of scenario forecasts to 2030 together with estimates of potential capacity gaps.

These results provide the main quantitative evidence used in the final technical report, policy summary and project presentation.

Chapter 13

References and AI Acknowledgement

13.1 References

Australian Academy of Science. 2024. "The Future Computing Needs of the Australian Science Sector." https://www.science.org.au/.

Australian Academy of Science. 2026. *Internship Project Brief: Forecasting Demand for National Research Supercomputing.*

National Computational Infrastructure. 2014a. *Annual Report 2013/14.* https://nci.org.au/sites/default/files/annual-reports/2019-07/About%20Us_Annual%20Reports_2013-14.pdf.

National Computational Infrastructure. 2014b. "NCMAS Information for Applicants." https://nci.org.au/news-events/news/ncmas-information-applicants.

National Computational Infrastructure. 2015. *Annual Report 2014/15.* https://nci.org.au/sites/default/files/annual-reports/2019-07/About%20Us_Annual%20Reports_2014-15.pdf.

National Computational Infrastructure. 2016. *Annual Report 2015/16.* https://nci.org.au/sites/default/files/annual-reports/2019-07/About%20Us_Annual%20Reports_2015-16.pdf.

National Computational Infrastructure. 2017. *Annual Report 2016/17.* https://nci.org.au/sites/default/files/annual-reports/2019-07/About%20Us_Annual%20Reports_2016-17.pdf.

National Computational Infrastructure. 2018a. *Annual Report 2017/18*. https://nci.org.au/sites/default/files/annual-reports/2019-07/About%20Us_Annual%20Reports_2017-18.pdf.

National Computational Infrastructure. 2018b. "NCMAS Allocations Now Available Online." https://nci.org.au/news-events/news/ncmas-allocations-now-available-online.

National Computational Infrastructure. 2019. *Annual Report 2018/19*. https://nci.org.au/sites/default/files/annual-reports/2019-11/NCI%20Report%202019%20web.pdf.

National Computational Infrastructure. 2020a. *Annual Report 2019/20*. https://nci.org.au/sites/default/files/annual-reports/2021-01/NCI%20Annual%20report%202019-20%20web_final_opt2.pdf.

National Computational Infrastructure. 2020b. *Compute Allocations 2019/20*. https://nci.org.au/sites/default/files/annual-reports/2021-01/Allocations%202019-20%20for%20website%20FINAL%20with%20pagenb.pdf.

National Computational Infrastructure. 2020c. *NCMAS 2020 Information for Applicants*. https://nci.org.au/sites/default/files/documents/2019-10/NCMAS-2020-InformationForApplicants.pdf.

National Computational Infrastructure. 2021. *Compute Allocations 2020–2021*. https://nci.org.au/sites/default/files/annual-reports/2021-12/NCI%20Compute%20Allocations%202020-2021_0.pdf.

National Computational Infrastructure. 2022. *NCMAS 2023 Information for Applicants*. https://my.nci.org.au/mancini-assets/media/mas/ncmas/2023/NCMAS2023-NCMAS_2023_Info4Applicants_v5-20220830T164026AEST.pdf.

National Computational Infrastructure. 2023. *NCMAS 2024 Information for Applicants*. https://my.nci.org.au/mancini-assets/media/mas/ncmas/2024/NCMAS2024-NCMAS_2024_-_Information_for_Applicants_20231109-20231109T131740AEDT.pdf.

National Computational Infrastructure. 2025. *NCMAS 2026 Information for Applicants*. https://my.nci.org.au/mancini-assets/media/mas/ncmas/2026/NCMAS2026-Information_for_Applicants-20251023T122535AEDT.pdf.

National Computational Infrastructure. n.d. "Who We Are." Accessed October 4, 2026. https://nci.org.au/.

National Computational Merit Allocation Scheme. 2021. "NCMAS 2021 Out-

comes." https://my.nci.org.au/mancini/ncmas/2021/outcomes.

National Computational Merit Allocation Scheme. 2022. "NCMAS 2022 Outcomes." https://my.nci.org.au/mancini/ncmas/2022/outcomes.

National Computational Merit Allocation Scheme. 2023. "NCMAS 2023 Outcomes." https://my.nci.org.au/mancini/ncmas/2023/outcomes.

National Computational Merit Allocation Scheme. 2024. "NCMAS 2024 Outcomes." https://my.nci.org.au/mancini/ncmas/2024/outcomes.

National Computational Merit Allocation Scheme. 2025. "NCMAS 2025 Outcomes." https://my.nci.org.au/mancini/ncmas/2025/outcomes.

National Computational Merit Allocation Scheme. 2026. "NCMAS 2026 Outcomes." https://my.nci.org.au/mancini/ncmas/2026/outcomes.

Pawsey Supercomputing Research Centre. 2015. *Annual Report 2014/15*. https://pawsey.org.au/wp-content/uploads/2018/10/ANNUALREPORT14-15v11web.pdf.

Pawsey Supercomputing Research Centre. 2017. *Annual Report 2016/17*. https://pawsey.org.au/wp-content/uploads/2018/10/Pawsey-Annual-Report-2016-2017.pdf.

Pawsey Supercomputing Research Centre. 2019. "Applications Open for 2019 Access to Pawsey Supercomputers." https://pawsey.org.au/applications-open-for-2019-access-to-pawsey-supercomputers/.

Pawsey Supercomputing Research Centre. n.d. "About Pawsey." Accessed October 4, 2026. https://pawsey.org.au/.

13.2 AI Acknowledgement

Generative AI tools, including ChatGPT, were used to support aspects of this project such as brainstorming, reviewing code, improving written expression, and assisting with the organisation of the analysis and documentation.

All data preparation, analytical decisions, model development, interpretation of results, and final conclusions were reviewed and verified by the author.

AI-generated suggestions were not treated as primary sources. Information and assumptions used in the analysis were checked against the original sources where appropriate.